NodeJS Introduction

Last Updated : 21 Feb, 2025

NodeJS is a runtime environment for executing **JavaScript** outside the **browser**, built on the **V8 JavaScript engine**. It enables **server-side development**, supports **asynchronous**, **event-driven programming**, and efficiently handles scalable network applications.

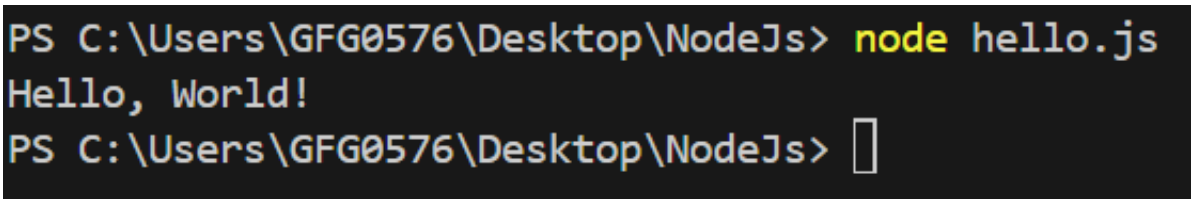
- NodeJS is **single-threaded**, utilizing an **event loop** to handle multiple tasks concurrently.
- It is **asynchronous** and **non-blocking**, meaning operations do not wait for execution to complete.
- The **V8 engine** compiles **JavaScript** to machine code, making **NodeJS** fast and efficient.

Hello, World!" Program in NodeJS

A "Hello, World!" program is the simplest way to get started with NodeJS. Unlike the browser, where **JavaScript** runs inside the console, **NodeJs** executes **JavaScript** in a server environment or via the command line.

```
console.log("Hello, World!");
```

Output:



Hello, World! in NodeJS

Key Features of NodeJS

- **Server-Side JavaScript:** NodeJS allows JavaScript to run outside the browser, enabling backend development.
- **Asynchronous & Non-Blocking:** Uses an event-driven architecture to handle multiple requests without waiting, improving performance.
- **Single-Threaded Event Loop:** Efficiently manages concurrent tasks using a single thread, avoiding thread overhead.
- **Fast Execution:** Powered by the V8 JavaScript Engine, NodeJS compiles code directly to machine code for faster execution.
- **Scalable & Lightweight:** Ideal for building microservices and handling high-traffic applications efficiently.
- **Rich NPM Ecosystem:** Access to thousands of open-source libraries through Node Package Manager (NPM) for faster development.

How NodeJS Works?

NodeJS is a **runtime environment** that allows JavaScript to run outside the **browser**. It is asynchronous, **event-driven,** and built on the **V8 JavaScript engine**, making it ideal for scalable network applications.

Single-Threaded Event Loop Model

NodeJS operates on a single thread but efficiently handles multiple concurrent requests using an event loop.

- **Client Sends a Request:** The request can be for data retrieval, file access, or database queries.
- **NodeJS Places the Request in the Event Loop:** If the request is non-blocking (e.g., database fetch), it is sent to a worker thread without blocking execution.
- **Asynchronous Operations Continue in Background:** While waiting for a response, NodeJS processes other tasks.
- **Callback Execution:** Once the operation completes, the callback function executes, and the response is sent back to the client.

```
const fs = require('fs');

fs.readFile('file.txt', 'utf8', (err, data) => {
```

```
    if (err) throw err;
    console.log(data);
});

console.log("Reading file...");
```

Components of NodeJS Architecture

- **V8 Engine:** Compiles JavaScript to machine code for fast execution.
- **Event Loop:** Manages asynchronous tasks without blocking the main thread.
- **Libuv:** Handles I/O operations, thread pool, and timers.
- **Non-Blocking I/O:** Executes tasks without waiting for previous ones to complete.

Where to Use NodeJS?

NodeJS is best suited for applications that require high performance, scalability, and real-time processing. Below are some common use cases:

- **Web APIs and Backend Services :** Ideal for building RESTful APIs and GraphQL APIs. It also Used in backend services for mobile apps and web applications.
- **Real-Time Applications:** Chat applications (e.g., WhatsApp, Slack). Live streaming services (e.g., Netflix, Twitch).
- **Microservices Architecture:** It helps in developing scalable and independent services. It also used in cloud-based applications.
- **IoT (Internet of Things) Applications:** Handles real-time data streaming from IoT devices. It is suitable for smart home automation and sensor-based systems.
- **Serverless Computing:** Works well with AWS Lambda, Azure Functions, and Google Cloud Functions. Runs lightweight serverless functions efficiently.
- **Single-Page Applications (SPAs):** It used in React, Angular, and Vue.js applications. It manages API requests efficiently in the backend.
- **Data-Intensive Applications:** It used for big data processing and real-time analytics. It works well with NoSQL databases like

MongoDB and Firebase.

Applications of NodeJS

Web Development: NodeJS powers backend services for web applications, handling HTTP requests and managing APIs efficiently.

- **Real-Time Applications:** Used in chat applications, **online gaming, and live streaming services** due to its **event-driven, non-blocking architecture.**
- **Server-Side Applications:** Enables full-stack JavaScript development, handling database operations, **authentication, and server logic.**
- **Microservices Architecture:** Helps build scalable, independent microservices for modern web applications.
- **API Development:** Ideal for creating RESTful and GraphQL APIs that interact with databases and client applications.
- **IoT Applications:** NodeJS efficiently handles real-time data processing for IoT devices like sensors and smart home systems.

Limitations of NodeJS

Security Risks – Being open-source and widely used, NodeJS applications are prone to security vulnerabilities like Cross-Site Scripting (XSS) and SQL Injection if not handled properly.

- **Single-Threaded Limitations:** While the event loop manages concurrency efficiently, CPU-intensive tasks can block the thread, affecting performance.
- **Performance Issues with Heavy Computation:** NodeJS is not ideal for CPU-bound tasks like machine learning or video processing, as it lacks multi-threading for heavy computations.
- **Callback Hell:** Older asynchronous code heavily relied on nested callbacks, making it difficult to read and maintain, though Promises and Async/Await help mitigate this.
- **Weak Type Checking:** Since NodeJS runs JavaScript, dynamic typing can lead to runtime errors and unpredictable behavior without strict type enforcement.

NodeJS Basics

Last Updated : 06 Mar, 2025

NodeJS is a powerful and efficient open-source runtime environment built on Chrome's V8 JavaScript engine. It allows developers to run JavaScript on the server side, which helps in the creation of scalable and high-performance web applications.

In this article, we will discuss the NodeJs Basics and its implementation.

What is NodeJS?

NodeJS is a **JavaScript runtime environment** built on Chrome's V8 JavaScript engine that allows developers to execute JavaScript code outside the browser. It can make console-based and web-based NodeJS applications. Some of the features of the NodeJs are mentioned below:

- **Non-blocking I/O**: NodeJS is asynchronous, enabling efficient handling of concurrent requests.
- **Event-driven architecture:** Developers can create event-driven applications using callbacks and event emitters.
- **Extensive module ecosystem:** npm (Node Package Manager) provides access to thousands of reusable packages.
- **Single-threaded Model:** Node js runs on a single thread despite being asynchronous, it uses an event loop to handle requests.

Setting Up NodeJS

To start using NodeJS, you'll first need to install it on your system.

Step 1: Download and Install NodeJS

- Go to the NodeJS official website or you can follow our article to install NodeJS
- Download the appropriate installer for your operating system (Windows, macOS, or Linux).
- Follow the installation steps to set up NodeJS.

Step 2: Verify Installation

Once installed, you can verify the installation by opening your terminal and typing the following commands:

```
node -v
npm -v
```

Step 3: Create a NodeJS Project

Create a new directory for your project and initialize it with npm by running.

```
mkdir node-project
cd node-project
```

This will generate a package.json file, which is essential for managing your project dependencies.

```
npm init -y
```

Step 4: Write Your First NodeJS Application:

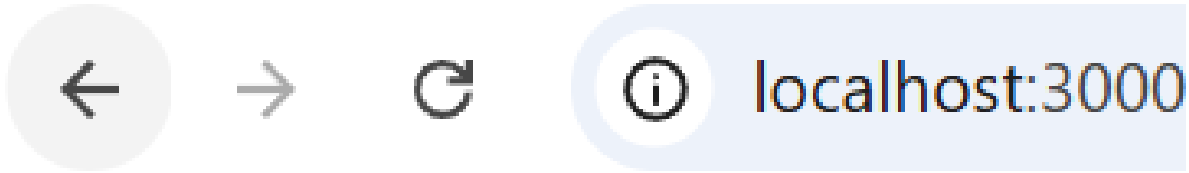
Create a new file called app.js and add the following code to create a simple HTTP server:

```
const http = require('http');

const server = http.createServer((req, res) => {
    res.write('Hello World!');
    res.end();
});

server.listen(3000, () => {
```

```
    console.log('Server running on port 3000');
});
```

Output:

```
node app.js
```



Hello World!

NodeJs

In this example

- The http module is imported using require('http') to allow you to create an HTTP server.
- http.createServer creates a new server that listens for incoming HTTP requests.
- The callback function (req, res) handles requests and responses.
- Inside the callback, res.write('Hello, World!') sends "Hello, World!" as a response, and res.end() ends the response.
- server.listen(3000) starts the server on port 3000. Once the server is running, it logs "Server running on port 3000" to the console.

NodeJS Architecture

NodeJS follows a **single-threaded event loop model** that handles all client requests using a single thread. It is based on a non-blocking I/O model, meaning the server can process multiple requests without waiting for one task to complete before starting the next.

- **Single-Threaded**: Although NodeJS runs on a single thread, it can handle thousands of concurrent requests using asynchronous callbacks.
- **Event-Driven**: The event-driven architecture allows NodeJS to process multiple connections efficiently by triggering callbacks when an event occurs (e.g., receiving data, file I/O completion).

- **Non-Blocking I/O**: Instead of waiting for I/O operations (like reading from a file or querying a database) to complete, NodeJS continues to execute the next line of code. The result is handled asynchronously, improving overall performance.

How Does NodeJS Work?

- **JavaScript Code**: The first step in the NodeJS process involves writing JavaScript code that defines the behavior of your server or application.
- **Parsing & Compilation:** The JavaScript code is parsed and compiled by the V8 JavaScript engine. The V8 engine compiles the code into machine code for fast execution.
- **Event Loop**: NodeJS uses the event loop to handle requests. It doesn't create a new thread for each request, but instead, it processes requests asynchronously using a single thread. This allows NodeJS to handle many requests without blocking.
- **Asynchronous Execution**: NodeJS uses asynchronous functions for tasks like reading from the filesystem, accessing databases, or making network requests. These functions do not block the execution of the rest of the program, which keeps the application responsive.
- **Call Back Functions:** In NodeJS, callbacks are used to handle asynchronous operations. Once an operation is completed, the corresponding callback is executed, continuing the flow of the program.
- **Event-driven Programming:** NodeJS applications can emit and listen to events using the events module. This allows for decoupling the application logic, leading to more maintainable and scalable systems.

Basic NodeJS Concepts

1. Modules in NodeJS

NodeJS is built around the concept of modules. Modules in NodeJS are reusable pieces of code that can be imported into your application. These can be built-in modules (like fs for file system operations, http for HTTP server, etc.) or external packages installed using NPM.

Common NodeJS Modules

- **HTTP Module:** The http module is used to create web servers. It allows you to handle requests and send responses.
- **FS (File System) Module**: The fs module provides an API to interact with the file system. It can be used to read and write files, check for file existence, etc.
- **Path Module:** The path module helps in handling and transforming file paths. It makes working with file systems easier and more cross-platform.
- **Event Module**: The events module allows objects to emit and listen to events, which helps in writing event-driven applications.
- **Express Framework:** While NodeJS provides basic capabilities, many developers use the Express framework, which simplifies routing, middleware integration, and HTTP request handling.

2. The Event Loop

NodeJS operates on a single-threaded, event-driven model. It uses an event loop to handle asynchronous operations. The event loop is a process that constantly checks if any asynchronous task (such as reading a file or making an HTTP request) has been completed and then invokes the appropriate callback function.

This allows NodeJS to handle many operations concurrently without blocking the main thread, making it efficient for I/O-heavy applications like web servers.

3. Asynchronous Programming

In NodeJS, many operations, such as reading files or accessing databases, are performed asynchronously. This means that the program doesn't wait for these operations to complete before moving on to the next one. Instead, it continues execution and provides a callback function that will be invoked once the operation finishes.

Advantages of Using NodeJS

1.1.2 Node.js Building Blocks

1. Global and Process Objects

✓ global Object:

It provides globally available variables and functions.

Examples:

- __dirname → gives the directory of the current module.
- __filename → gives the filename of the current module.
- setTimeout, clearInterval, etc., are part of global.

✓ process Object:

Represents the current Node.js process.

Key uses:

- process.env → access environment variables.
- process.argv → array of command-line arguments.
- process.exit(code) → ends the process with an optional code.
- process.on('exit', callback) → attach cleanup functions.

2. Buffers

Buffers are used to handle raw binary data.

You don't use strings for image, video, or file manipulation — you use buffers.

Created using:

```
const buffer = Buffer.from('Hello');
console.log(buffer); // <Buffer 48 65 6c 6c 6f>
```

You can read or write into a buffer directly.

Common in file handling and network communication.

3. Typed Arrays

Typed arrays store raw binary data using JavaScript ArrayBuffer.

Types: Int8Array, Uint8Array, Float32Array, etc.

Useful in:

Multimedia apps (audio/video).

Performance-sensitive tasks like gaming or graphics.

```
const intArray = new Int16Array([10, 20, 30]);
console.log(intArray[0]); // 10
```

4. Strings

Regular JavaScript strings, used to store and display text.

Often need to be encoded/decoded when interacting with Buffers.

Encoding types: utf8, ascii, base64, etc.

Example:

```
const buffer = Buffer.from('Hello', 'utf8');
console.log(buffer.toString()); // Hello
```

5. Streams

Streams help in handling large data without loading all at once.

Types:
Readable – e.g., reading files.
Writable – e.g., writing to a file.
Duplex – both readable and writable.
Transform – modify data as it passes through.

Example:

```
const fs = require('fs');
const readStream = fs.createReadStream('file.txt');
readStream.on('data', chunk => console.log(chunk.toString()));
```

6. Callbacks and Asynchronous
✅ Callback:
A function passed into another function to be called later.
Used when an operation (like reading a file) takes time.

✅ Asynchronous:
Non-blocking — lets other code run while waiting.
Node.js uses this for file handling, API calls, etc.

Example:

```
fs.readFile('text.txt', (err, data) => {
  if (err) return console.error(err);
  console.log(data.toString());
});
```

⚙️ Node.js Event Handling

7. Event Queue
Keeps track of events/tasks to be processed.
Node handles tasks like file reading or timers by putting them in this queue.

8. Event Emitter
Built into Node (require('events')).
Lets you create, trigger, and listen to custom events.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('msg', () => console.log('Message received'));
emitter.emit('msg');
```

9. Event Loop
The brain of Node.js!
It watches the Event Queue and processes tasks one by one.
It picks up finished async tasks (from callbacks, timers, etc.) and runs them.

10. Timers
setTimeout: Runs code once after a delay.
setInterval: Repeats code after a delay.
setImmediate: Runs after current code but before other I/O events.

Example:

```
setTimeout(() => console.log('Delayed message'), 2000);
```

11. Nested Callbacks
Occurs when callbacks are written inside other callbacks.
Looks messy:

```
fs.readFile('file1.txt', (err, data1) => {
  fs.readFile('file2.txt', (err, data2) => {
    fs.readFile('file3.txt', (err, data3) => {
      // ...
    });
  });
});
```

1.2 Exception Handling in Node.js

What is Exception Handling?
Exception handling is a way to catch and manage errors in a program so it doesn't crash and can handle problems gracefully.

🔷 Why It's Important in Node.js:
Node.js is asynchronous and non-blocking, so it's important to handle errors properly — especially in callbacks, promises, and async code.

🔹 1. Using try...catch
Used for synchronous (blocking) code. — where errors happen instantly.
It helps prevent your app from crashing when something goes wrong.

```
try {
    let a = 10;
    let b = a / 0;
    console.log(b);
} catch (err) {
    console.error('Error caught:', err.message);
}
```

🧠 Use when code might throw an error immediately.

🔹 2. Handling Errors in Callbacks
✅ Use for: Functions that follow the error-first callback pattern (like fs.readFile).
✅ Why: Node.js uses callbacks a lot for async tasks like file reading.

```
const fs = require('fs');

fs.readFile('file.txt', 'utf8', (err, data) => {
    if (err) {
        console.error('Error reading file:', err.message);
        return;
    }
    console.log(data);
});
```

📌 Note: Always check the err argument first before using the data.

🔹 3. Using Promises with .catch()
✅ Use for: Asynchronous operations that return a promise (like API calls, DB queries).
✅ Why: Keeps your code clean and makes error handling easier.

```
fetchData()
    .then(data => console.log(data))
    .catch(err => console.error('Error:', err.message));
```

📌 Note: If anything fails in the .then() chain, the .catch() will handle it.

🥢 4. Using async/await with try...catch

✔️ Use for: Writing asynchronous code in a more readable way (like synchronous code).

✔️ Why: Makes async code easier to understand and maintain.

```
async function getData() {
    try {
        const result = await fetchData();
        console.log(result);
    } catch (err) {
        console.error('Caught error:', err.message);
    }
}
```

📌 Note: You still need try...catch to catch any errors from the await.

🥢 5. Global Error Handling

✔️ Use for: Catching unexpected or uncaught errors.

✔️ Why: Prevents app from crashing due to unhandled exceptions.

```
process.on('uncaughtException', (err) => {
    console.error('Uncaught Exception:', err.message);
});
```

📌 Note: This should be your last safety net, not a replacement for proper error handling.

Node.js - REPL Terminal

The Node.js provides a **REPL** terminal, that allows to execute the JavaScript code interactively. It is an essential tool for testing, debugging and experimenting with JavaScript and Node.js features. The REPL is used to check the JavaScript functionality without setting up an entire development environment.

What is REPL

It is an interactive programming environment that allows users to run the JavaScript code directly in the terminal, It stands for:

- **Read** – It reads the user input and analyses it into the JavaScript expressions.
- **Eval** – It evaluates the analyzed JavaScript expressions.
- **Print** – It prints the evaluation result.
- **Loop** – Loops back to read the next input.

The REPL environment is automatically available, when we install the Node.js making it convenient for quick testing and debugging.

Starting the REPL

The REPL can be accessed directly from the command line. This makes it easy to write and test the JavaScript code without creating the separate script files. For launching the REPL terminal, open the command prompt or terminal and type:

```
node
```

This result in the REPL prompt:

```
Welcome to Node.js v22.13.1.
Type ".help" for more information.
>
```

Using REPL

The REPL terminal provides a environment, where users can execute the JavaScript expressions, perform calculations, declare variables and define the functions interactively

Running JavaScript Expressions

One of the key features of REPL is it ability to execute the JavaScript expressions on the go. It allows users to quickly test the JavaScript syntax, mathematical calculations, and function outputs without the need of script file.

Let's look at the simple example of executing the JavaScript expressions in REPL.

```
> 2 + 3
5
> 11 - 22
-11
> console.log("Welcome to TP.!")
Welcome to TP.!
undefined
```

The REPL evaluates the expressions and prints the result immediately, making it as a good tool for rapid experimentation.

Variable Declarations

In REPL, we can declare variables dynamically using the **var**, **let** or **const**. It allows users to experiment with different variable scopes and behaviour without the need of separate script file. Variables declared in the REPL continue throughout the session, making it useful for calculations and debugging.

```
> let a = 1121;
undefined
> a * 2
2242
```

Multiline Code Execution

It supports executing the multi-line JavaScript code, which is particularly useful for defining functions, loops and complex statements that requires multi lines.

When we try to enter an incomplete expression, REPL automatically enters multiline mode, allowing to finish the incomplete code. It allows the users to test more structured and logical code within the REPL session.

```
> function x(a){
... return "Welcome To, " +a;
... }
undefined
> x("TutorialsPoint")
'Welcome To, TutorialsPoint'
```

Using Underscore

The REPL automatically stores the result of the last evaluated expression in a special variable underscore(_). It allows the users to reuse the last output without retyping the entire expression. Let's consider the simple example for understanding better

```
> 112 + 234
346
> _ / 2
173
```

Here, the _ variable holds the value 346, and then it is divided by 2 to get 173.

Working with Global Objects

The Node.js REPL provides the direct access to the global objects, making it easy for the users to interact with system-level functionalities.

Global objects in Node.js such as process, console and Buffer can be accessed without requiring the additional imports. These objects provides the essential capabilities like handling system information, managing buffers.

Process
For example, we are going to consider the process object which provides the information about the current Node.js runtime.

```
> console.log(process.version)
v22.13.1
```

Buffer

Now, we are going to use the Buffer object, which allows to manipulate the binary data.

```
> Buffer.from('TutorialsPoint')
<Buffer 54 75 74 6f 72 69 61 6c 73 50 6f 69 6e 74>
```

Handling Errors

The REPL provides the meaningful error message to help the users to debug the issues. It is particularly useful for testing the small code snippets before dumping them into a larger application. Common errors like syntax errors, type errors and reference errors can be handled by detailed feedback.

```
> let y = 123
undefined
> y()
Uncaught TypeError: y is not a function
```

REPL Commands

The Node.js REPL includes the several built-in commands for better usability, some of them are listed below:

S.No	Dot Commands & Description
1	**.exit** It exits the REPL session.
2	**.clear** It clears the REPL environment.
3	**.save** It saves the REPL session to a file.
4	**.load** It loads a file into the REPL session.
5	**.editor** It enters the multiline editor mode.

Unit 2: Node Modules and Node Package Manager (NPM)

2.1 Overview of Node Module System

✅ 1. What is a Module?

- A module in Node.js is like a small block of code that does a specific task.
- Think of it as a reusable file or function that you can use in different parts of your app.

✅ 2. Why Use Modules?

- Keeps code clean, organized, and reusable.
- You can break big programs into smaller files/modules.
- Helps in collaborative development (teamwork).

✅ 3. Types of Modules in Node.js

Type	Description	Example
1. Built-in Modules	Provided by Node.js itself	fs, http, path
2. User-defined Modules	Created by us	myModule.js
3. Third-party Modules	Installed via npm	express, mongoose

✅ 4. Exporting a Module (User-defined)

💼 File: math.js

```
function add(a, b) {
   return a + b;
}

module.exports = add;
```

✅ 5. Importing a Module

💼 File: app.js

```
const add = require('./math');
console.log(add(5, 3)); // Output: 8
```

✅ 6. Built-in Module Example

```
const os = require('os');
console.log(os.platform());    // e.g. win32
console.log(os.freemem());     // free memory in bytes
```

✅ 7. Third-party Module Example

```
npm install express
const express = require('express');
const app = express();
```

✅ 8. Conclusion

- The Node Module System is the backbone of Node.js apps.

- It allows you to organize code and use built-in or third-party tools easily.
- Using require() and module.exports, you can share and reuse code across files.
- Let me know if you'd like this in PDF format, Word file, or handwritten-style!

2.2 Overview of Node Package Manager (NPM)

✅ 1. What is NPM?

- NPM stands for Node Package Manager.
- It is the default package manager for Node.js.
- It helps developers install, share, and manage packages (also called modules or libraries).

✅ 2. What is a Package?

- A package is a collection of code files that solve a particular problem.
- Example: express (for web server), mongoose (for MongoDB), lodash (for utility functions).

✅ 3. Why Use NPM?

- To reuse code written by others.
- To save time in development.
- To organize project dependencies.

✅ 4. Common NPM Commands

Command	Purpose
npm init	Create a package.json file
npm install package-name	Install a package
npm uninstall package-name	Remove a package
npm update	Update installed packages
npm list	Show installed packages

✅ 5. What is package.json?

- A file that stores project info and dependencies.
- Automatically created using npm init.

```
{
  "name": "my-app",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

✅ 6. Global vs Local Packages

Type	Installed Location	Example
Local	Inside your project folder	npm install express
Global	System-wide installation	npm install -g nodemon

✅ 7. Online NPM Registry
All packages are available at:
🌐 https://www.npmjs.com

✅ 8. Conclusion
- NPM is an essential tool for Node.js developers.
- It makes it easy to manage packages, share code, and maintain projects efficiently.
- Without NPM, development would be much slower and harder.

2.3 Overview of Node Version Manager

✅ 1. What is NVM (Node Version Manager)?
- NVM stands for Node Version Manager.
- It is a tool that allows you to install, switch, and manage different versions of Node.js on your system.

✅ 2. Why Use NVM?
- Sometimes, different projects need different versions of Node.js.
- NVM lets you easily switch between versions without uninstalling and reinstalling.

🔽 Example:
- Project A needs Node.js v14
- Project B needs Node.js v18

👉 With NVM, you can switch between them anytime.

✅ 4. Common NVM Commands

Command	Description
nvm install 18	Install Node.js version 18
nvm install 14	Install Node.js version 14
nvm use 14	Switch to Node.js version 14
nvm ls	List all installed Node versions
nvm current	Show the currently active version
nvm uninstall 14	Remove version 14

✅ 5. Real-Life Use Case
- You're working on an old project that only works in Node.js v10.
- Your new project uses Node.js v20.

☞ Use nvm to switch between versions with a single command.

✓ 6. Benefits of NVM

- Saves time
- Avoids conflicts
- Helpful for developers working on multiple projects
- Keeps system clean and organized

✓ 7. Example:

```
nvm install 16
nvm use 16
node -v    // Output: v16.x.x
```

✓ 8. Conclusion

- Node Version Manager (NVM) is a helpful tool for managing multiple Node.js versions on one system.
- It is essential for developers who work on different projects with different Node.js requirements.

2.4 Creating and Publishing Node Modules

✓ 1. What is a Node Module?

- A Node module is a reusable JavaScript file that contains some logic or functions.
- You can create your own module and also publish it to npm (Node Package Manager) so others can use it.

✓ 2. Why Create a Module?

- Reuse your code in multiple projects.
- Share helpful functions with the developer community.
- Keep your project clean and organized.

✓ 3. Steps to Create a Node Module

🔹 Step 1: Create a Folder

```
mkdir mymodule
cd mymodule
```

🔹 Step 2: Initialize npm

```
npm init
```

☞ This creates a package.json file to store module info.

🔹 Step 3: Create the Module File

💼 File: index.js

```
function sayHello(name) {
   return `Hello, ${name}!`;
}

module.exports = sayHello;
```

✅ 4. Test Your Module Locally

📁 File: test.js

```
const greet = require('./index');
console.log(greet("Rishi"));    // Output: Hello, Rishi!
```

✅ 5. Create npm Account (One Time Only)

- Go to https://www.npmjs.com/ and sign up.
- Then login using terminal:

```
npm login
```

✅ 6. Publish the Module

```
npm publish
```

📌 After publishing, your module will be live on:

👉 https://www.npmjs.com/package/mymodule

✅ 7. Install Published Module (in any project)

```
npm install mymodule
```

Then use it in your code:

```
const greet = require('mymodule');
console.log(greet("Rishi"));
```

✅ 8. Conclusion

- Creating and publishing Node modules lets you reuse and share your code.
- It involves simple steps like writing code, creating package.json, and using npm publish.
- Other developers can now use your module with npm install.

2.5 Node Module – Async

✅ 1. What is the Async Module?

- The async module in Node.js helps you manage asynchronous (non-blocking) operations.
- It makes it easy to run tasks one after another or at the same time without writing confusing callback code.

✅ 2. Why Use Async Module?

- Node.js is non-blocking by default.
- When you have many tasks like reading files, calling APIs, or database operations, async

helps you:

- Run tasks in order (like a to-do list)
- Run tasks together (in parallel)
- Handle errors more easily

✅ 3. How to Install Async?

npm install async

✅ 4. Example: Running Tasks in Series

```
const async = require('async');

async.series([
   function(callback) {
      setTimeout(() => {
         console.log("Task 1 done");
         callback(null, 'One');
      }, 1000);
   },
   function(callback) {
      console.log("Task 2 done");
      callback(null, 'Two');
   }
], function(err, results) {
   console.log(results);    // ['One', 'Two']
});
```

🔼 Here, Task 1 runs first, then Task 2.

✅ 5. Useful Functions in Async

Function	Purpose
series()	Runs tasks one after another
parallel()	Runs all tasks at the same time
waterfall()	Passes result of one to the next
each()	Loops through array asynchronously

✅ 6. Conclusion

- The async module is very helpful in writing clean, non-blocking, and organized code in Node.js.
- It avoids the problem of callback hell and makes code easy to manage when working with multiple async tasks.

Node Modules – Commander and Underscore

✅ 1. Commander Module

🔷 What is Commander?

- Commander is a Node.js module used to build command-line applications (CLI tools).
- It helps you create commands, options, and flags like --help, --version, --name, etc.

🔷 Why Use Commander?

- Easy to build custom terminal tools.
- Adds user-friendly features like help messages and argument parsing.

🔷 How to Install?

```
npm install commander
```

🔷 Example:

```
const { program } = require('commander');

program
  .version('1.0.0')
  .option('-n, --name <type>', 'Enter your name');

program.parse(process.argv);

const options = program.opts();
console.log(`Hello, ${options.name}`);
```

✅ Run it in terminal:

```
node app.js --name Rishi
```

🟢 Output: Hello, Rishi

🔷 Key Features of Commander:

Feature	Use
.version()	Sets CLI version
.option()	Adds options like --name
.parse()	Parses input from terminal
.help()	Shows help menu

2. Underscore Module

🔷 What is Underscore?

- Underscore.js is a utility library for arrays, objects, and functions.
- It provides ready-made functions to make code shorter and cleaner.

🔷 Why Use Underscore?

- Helps with data filtering, sorting, mapping, etc.
- Reduces the need for writing long custom functions.

🔹 How to Install?

npm install underscore

🔹 Example:

```
const _ = require('underscore');

const users = [
   { name: "Rishi", age: 22 },
   { name: "Amit", age: 18 }
];

const youngUsers = _.filter(users, user => user.age < 21);
console.log(youngUsers);
```

🟢 Output:

```
[ { name: 'Amit', age: 18 } ]
```

🔹 Common Underscore Functions:

Function	Description
_.each()	Loop through arrays/objects
_.map()	Modify all items in an array
_.filter()	Get items that match a condition
_.find()	Find the first matching item
_.where()	Get all objects that match criteria

✅ Conclusion

Module	Use Case
Commander	Building custom CLI tools (like npm, git)
Underscore	Simplifying code for data handling and logic

- Both modules help make your Node.js applications powerful and easier to manage.
- Commander focuses on command-line interaction, while Underscore is about clean data processing.

Node Module – OAuth

✅ 1. What is OAuth?

- OAuth (Open Authorization) is a secure way to log in users using third-party services like Google, Facebook, GitHub, etc.

- It allows users to authorize apps without sharing passwords.

✅ 2. Why Use OAuth in Node.js?

- Instead of creating your own login system, you can let users log in using trusted platforms.
- Example: “Log in with Google” button on many websites.

✅ 3. Common Use in Node.js

- OAuth is commonly used with the passport module (Passport.js), along with strategies like:
  - passport-google-oauth20
  - passport-facebook

✅ 4. Example: Google Login with Passport & OAuth

```
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;

passport.use(new GoogleStrategy({
      clientID: 'GOOGLE_CLIENT_ID',
      clientSecret: 'GOOGLE_CLIENT_SECRET',
      callbackURL: '/auth/google/callback'
   },
   function(accessToken, refreshToken, profile, done) {
      // Save user info here
      return done(null, profile);
   }
));
```

👉 After setting this, users can log in using Google.

✅ 5. Benefits of OAuth:

Benefit	Description
🔒 Secure Login	No need to store passwords
🧠 Easy for users	One-click login with Google/Facebook
🔄 Reusable Identity	Same login for multiple apps

✅ 6. Conclusion

- OAuth in Node.js is used to authenticate users via third-party services.
- It provides security, convenience, and is often used with Passport.js.
- Very useful in modern web and mobile applications.

2.6 Overview of Other Utility Modules in Node.js

✅ What are Utility Modules?

- Utility modules in Node.js are built-in tools that help perform common tasks like handling file paths, system info, events, formatting, etc.
- These modules are part of Node.js — no need to install separately using npm.

✅ Important Utility Modules

🔹 1. path Module

- Helps work with file and folder paths in a cross-platform way.
- Automatically handles slashes (/, \) depending on Windows or Linux.

```
const path = require('path');
console.log(path.join(__dirname, 'folder', 'file.txt'));
```

✅ Use: Creating and working with file system paths.

🔹 2. os Module

- Provides information about the operating system.

```
const os = require('os');
console.log(os.platform());    // OS type (e.g., 'win32')
console.log(os.freemem());     // Free memory in bytes
```

✅ Use: Get system data like memory, CPU, OS type, etc.

🔹 3. util Module

- Contains helper functions to make coding easier.
- Includes tools like promisify(), format(), and debuglog().

```
const util = require('util');
console.log(util.format('Hello %s', 'Rishi')); // Hello Rishi
```

✅ Use: Formatting strings, converting callbacks to promises.

🔹 4. events Module

- Allows us to create and handle custom events.

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

myEmitter.on('greet', () => {
   console.log('Hello from event!');
});

myEmitter.emit('greet');
```

✅ Use: Making your app event-driven.

🔹 5. querystring Module

- Helps to parse and format URL query strings.

```
const qs = require('querystring');
```

```
const result = qs.parse('name=Rishi&age=21');
console.log(result);    // { name: 'Rishi', age: '21' }
```

✅ Use: Useful in web apps for handling URLs.

✅ Conclusion

Module	Purpose
path	File and folder path handling
os	Get system info like memory, CPU
util	Helper functions for formatting, promises
events	Create and handle custom events
querystring	Parse and build query strings
assert	Testing and validation

Unit 3: Node with the Local System and the Web

3.1 Streams and Pipes

✅ What is a Stream?

- A stream is a way to read or write data in small pieces (chunks) instead of loading everything at once.
- This is useful for large files or continuous data (like videos, audio, or network).

🧠 Example: Watching a YouTube video while it's still loading is like a stream — you don't need the whole file before watching.

✅ Types of Streams in Node.js

Type	Description
Readable	For reading data (e.g., file reading)
Writable	For writing data (e.g., file writing)
Duplex	Can read and write (e.g., TCP sockets)
Transform	Modify data while streaming (e.g., zip)

✅ Readable Stream Example:

```
const fs = require('fs');
const readStream = fs.createReadStream('input.txt');

readStream.on('data', (chunk) => {
   console.log('Received chunk:', chunk.toString());
});
```

📄 Reads the file input.txt chunk by chunk.

✅ Writable Stream Example:

```
const fs = require('fs');
const writeStream = fs.createWriteStream('output.txt');

writeStream.write('Hello, this is a stream!');
writeStream.end();
```

📄 Writes text to output.txt file.

✅ What is a Pipe?

- A pipe is a shortcut to connect a readable stream to a writable stream.
- Pipes handle data flow automatically — no need for manual events.

✅ Pipe Example: Copy File Using Stream

```
const fs = require('fs');
```

```
const read = fs.createReadStream('input.txt');
const write = fs.createWriteStream('output.txt');

read.pipe(write);
```

📄 This copies the contents of input.txt into output.txt.

✅ Why Use Streams and Pipes?

Feature	Benefit
💨 Faster	No need to load all data at once
🧠 Memory Efficient	Works well with large files or data
🔄 Non-blocking	Continues other tasks while streaming
🔌 Easy Handling	Pipes make connecting streams simple

✅ Conclusion

- Streams let you process data in chunks (like reading or writing files).
- Pipes make it easy to connect streams together.
- They are fast, memory-efficient, and great for real-time applications.

3.2 Node.js and the File System – fs.Stats Class

✅ 1. What is the fs module?

- fs stands for File System module in Node.js.
- It allows you to read, write, update, and delete files and folders on your computer.

✅ 2. What is fs.Stats class?

- fs.Stats is a class in the fs module that contains information (metadata) about a file or directory.
- It tells you things like:
    - Is it a file or folder?
    - File size
    - Last modified time
    - Created time
    - Permissions, etc.

✅ 3. How to Get fs.Stats Info

You get fs.Stats using fs.stat() or fs.lstat():

```
const fs = require('fs');

fs.stat('example.txt', (err, stats) => {
   if (err) throw err;

   console.log(stats); // This is a fs.Stats object
```

```
});
```

✅ 4. Common fs.Stats Properties and Methods

Property / Method	Description
stats.isFile()	Returns true if it's a file
stats.isDirectory()	Returns true if it's a folder
stats.size	Size of the file in bytes
stats.mtime	Last modified time (Date)
stats.birthtime	Created time (Date)
stats.mode	Permissions info

✅ 5. Example: Check if it's a File or Folder

```
fs.stat('example.txt', (err, stats) => {
    if (stats.isFile()) {
        console.log('It is a file.');
    } else if (stats.isDirectory()) {
        console.log('It is a directory.');
    }
});
```

✅ 6. Why is fs.Stats Useful?

- Helps check file size, type, last modified date, etc.
- Useful for file uploads, backup systems, logs, or file explorers.

Node.js and the File System – File System Watcher

✅ 1. What is a File System Watcher?

- A File System Watcher lets you monitor files or folders for any changes like:
    - File created
    - File modified
    - File deleted
- In Node.js, this is done using fs.watch() from the fs (File System) module.

✅ 2. Why Use It?

- To automatically detect changes in files or directories.
- Useful for:
    - Live-reloading apps (like web servers)
    - Watching logs
    - Monitoring uploads or config files

✅ 3. How to Use fs.watch()

```
const fs = require('fs');

fs.watch('myFile.txt', (eventType, filename) => {
  console.log(`File ${filename} was ${eventType}`);
});
```

📝 This code watches myFile.txt and logs when it's changed or renamed.

✅ 4. Parameters of fs.watch()

Parameter	Description
filename	Name of the file or directory to watch
eventType	Event like 'rename' or 'change'
listener	Callback function when event happens

✅ 5. Example Output

If you change the file, it may show:

```
File myFile.txt was change
```

If you rename the file:

```
File myFile.txt was rename
```

✅ 6. Limitations

- May not be 100% reliable on all operating systems (especially for deep folder watching).
- For advanced needs, people use external modules like chokidar.

✅ Conclusion

- fs.watch() is a built-in Node.js feature to monitor files and folders for real-time changes.
- It’s useful in applications that need to react automatically to file updates.

Node.js and the File System – File Read and Write

✅ 1. What is the fs Module?

- fs stands for File System.
- It is a built-in Node.js module used to read from and write to files on your computer.
- You must import it like this:

```
const fs = require('fs');
```

✅ 2. Reading a File (Synchronous & Asynchronous)

🔹 A. Asynchronous File Read (Non-blocking)

```
fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

📌 Explanation:

- 'example.txt' is the file to read.
- 'utf8' means read the file as text.
- data contains the file content.
- Non-blocking → other code can run while reading.

🔹 B. Synchronous File Read (Blocking)

```
const data = fs.readFileSync('example.txt', 'utf8');
console.log(data);
```

📌 Explanation:

- This blocks the program until the file is fully read.
- Good for simple tasks, not recommended in big apps.

✅ 3. Writing to a File

🔹 A. Asynchronous File Write

```
fs.writeFile('output.txt', 'Hello, Rishi!', (err) => {
  if (err) throw err;
  console.log('File written successfully!');
});
```

📌 Explanation:

- Creates output.txt and writes text.
- If file exists, it overwrites.
- Callback is called after writing is done.

🔹 B. Synchronous File Write

```
fs.writeFileSync('output.txt', 'Hello again!');
console.log('Written synchronously!');
```

📌 Explanation:

Blocks the code until writing is complete.

✅ 4. Appending Data to a File

```
fs.appendFile('output.txt', '\nThis is new content!', (err) => {
  if (err) throw err;
  console.log('Content added!');
});
```

📌 Adds data at the end of the file without deleting old content.

✅ 5. Error Handling Example

```
fs.readFile('nofile.txt', 'utf8', (err, data) => {
  if (err) {
    console.log('File not found!');
  } else {
    console.log(data);
```

```
    }
});
```
✅ Always check for errors while reading or writing files.

Node.js and the File System – Directory Access and Maintenance

✅ 1. What is a Directory?

- A directory is just a folder that contains files or other folders.
- Node.js lets you create, read, delete, and check directories using the fs module.

✅ 2. Create a Directory

```
const fs = require('fs');

fs.mkdir('myFolder', (err) => {
    if (err) throw err;
    console.log('Folder created!');
});
```
📌 This creates a folder named myFolder.
You can also use fs.mkdirSync() for synchronous creation.

✅ 3. Read a Directory

```
fs.readdir('myFolder', (err, files) => {
    if (err) throw err;
    console.log('Files in folder:', files);
});
```
📌 This shows the list of files/folders inside myFolder.

✅ 4. Check if a Directory Exists

```
if (fs.existsSync('myFolder')) {
    console.log('Folder exists!');
} else {
    console.log('Folder does not exist!');
}
```
📌 Helps in avoiding errors before reading or deleting a folder.

✅ 5. Delete a Directory

```
fs.rmdir('myFolder', (err) => {
    if (err) throw err;
    console.log('Folder deleted!');
});
```
📌 Deletes the folder, but it must be empty.
Use fs.rm() or fs.rmdirSync() in newer versions of Node.js.

✅ 6. Create Directory with Files

```
fs.mkdir('testDir', () => {
    fs.writeFile('testDir/hello.txt', 'Hello!', () => {
        console.log('Folder and file created!');
    });
});
```

📌 Automatically creates a folder and adds a file inside it.

✅ 7. Use Case: Maintenance Tasks

Task	Purpose
✅ Create backup folder	To store backup files
🧹 Clean up empty folders	Free space or reset logs
📁 Log directory check	Ensure log folder exists before logging
📄 List contents	Show files in a web app

✅ 8. Synchronous vs Asynchronous

- fs.mkdirSync(), fs.rmdirSync(), fs.readdirSync() are blocking.
- fs.mkdir(), fs.rmdir(), fs.readdir() are non-blocking (better for apps).

Node.js and the File System – File Streams

✅ 1. What is a File Stream?

- A file stream is a way to read or write data from/to a file in chunks (small parts) instead of all at once.
- Very useful for large files — saves memory and improves performance.

🧠 Think of it like watching a movie online — you don't need to download the full file before watching. That's streaming!

✅ 2. Why Use File Streams?

Feature	Benefit
🎁 Chunk-based	Works with large files easily
⚡ Fast	Starts processing without full file
💾 Memory-safe	Uses less RAM
🔄 Non-blocking	Doesn't freeze your app

✅ 3. Types of Streams

Node.js supports 4 types of streams:

- Readable – Read data from a source (e.g., file)
- Writable – Write data to a destination (e.g., file)
- Duplex – Read and write (e.g., socket)
- Transform – Modify data while streaming (e.g., compression)

✅ 4. Read File Using Stream

```
const fs = require('fs');

const readStream = fs.createReadStream('example.txt', 'utf8');

readStream.on('data', (chunk) => {
    console.log('Received chunk:', chunk);
});
```

📌 Reads file example.txt chunk by chunk and prints each part.

✅ 5. Write File Using Stream

```
const fs = require('fs');

const writeStream = fs.createWriteStream('output.txt');

writeStream.write('Hello, this is a streamed write!');
writeStream.end();
```

📌 Writes data to output.txt using a write stream.

✅ 6. Pipe for Easy Stream Handling

```
const fs = require('fs');

const read = fs.createReadStream('input.txt');
const write = fs.createWriteStream('output.txt');

read.pipe(write);
```

📌 This copies input.txt to output.txt using piping, which connects readable and writable streams directly.

✅ Conclusion

- File streams in Node.js allow efficient and fast handling of files.
- They are perfect for large files, real-time processing, or copying data.
- Use fs.createReadStream() and fs.createWriteStream() to work with them.

3.3 Resource Access with Path

✅ 1. What is the path module?

- path is a built-in module in Node.js.
- It helps you work with file and folder paths easily.
- It makes your code cross-platform (works on Windows, Mac, Linux).

✅ 2. Why is path important?

- Avoids errors with slashes (/ or \) in file paths.
- Helps build correct paths when accessing files, folders, or resources.
- Ensures code runs the same on all operating systems.

✅ 3. Load the Path Module

```
const path = require('path');
```

✅ 4. Common path Methods

Method	Description
path.join()	Joins path parts correctly
path.resolve()	Resolves a full absolute path
path.basename()	Gets file name from full path
path.dirname()	Gets folder name from full path
path.extname()	Gets file extension (like .txt, .js)

✅ 5. Example: Access a File Safely

```
const filePath = path.join(__dirname, 'data', 'info.txt');
console.log(filePath);
```

📌 __dirname is the current folder.

📌 This gives a full path like:

C:\Users\Rishi\project\data\info.txt

✅ Works on any OS!

✅ 6. Example: Get File Info

```
const file = '/user/docs/file.txt';

console.log(path.basename(file));    // file.txt
console.log(path.dirname(file));     // /user/docs
console.log(path.extname(file));     // .txt
```

✅ 7. Real-Life Uses

- Reading files from folders (fs.readFile(path.join(...)))
- Serving images, PDFs, or videos in a web server
- Uploading or downloading files
- Managing file names and extensions

Unit 4: Node and Web Application

HTTP Module in Node.js

✅ 1. What is the HTTP Module?

- The http module is a built-in module in Node.js.
- It allows Node.js to create a web server and handle HTTP requests and responses.
- No need to install anything — it comes with Node.js.

✅ 2. Why Use the HTTP Module?

- To create custom web servers
- To handle client requests (like from browsers)
- To serve HTML, JSON, or files over the internet or locally

✅ 3. How to Use the HTTP Module?

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.write('Hello, Rishi!');
  res.end();
});

server.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

📌 This creates a basic server on port 3000.

✅ 4. Common HTTP Methods

Method	Use
GET	Get data from server
POST	Send data to server
PUT	Update data
DELETE	Delete data

You can check the request method using req.method.

✅ 5. Serving Different Pages

```
const http = require('http');

http.createServer((req, res) => {
  if (req.url === '/') {
    res.end('Home Page');
  } else if (req.url === '/about') {
    res.end('About Page');
```

```
    } else {
        res.end('Page Not Found');
    }
}).listen(3000);
```

📌 This checks the URL and sends different responses.

✅ 6. Real-Life Uses

- Making your own backend without frameworks
- Creating REST APIs
- Serving HTML, JSON, or file content
- Handling routes and user requests

✅ Conclusion

- The http module in Node.js is used to build web servers.
- It handles requests, responses, and supports all HTTP methods.
- It's simple, powerful, and doesn't need external packages.

Path Module in Node.js

✅ 1. What is the Path Module?

- The path module is a built-in Node.js module.
- It helps to work with file and folder paths easily.
- Useful for creating file paths that work on all operating systems.

🧠 Example: Windows uses \, Linux uses /. The path module handles this automatically.

✅ 2. How to Use the Path Module?

```
const path = require('path');
```

✅ 3. Commonly Used path Methods

Method	Description
path.join()	Joins multiple parts into one full path
path.resolve()	Gives absolute path
path.basename()	Gets the file name from a path
path.dirname()	Gets the folder name from a path
path.extname()	Gets the file extension (like .txt, .js)

✅ 4. Examples

```
const path = require('path');

// Join folders and file
console.log(path.join('folder', 'subfolder', 'file.txt'));
```

```
// Output: folder/subfolder/file.txt (works on all OS)

console.log(path.basename('/user/docs/file.txt')); // file.txt
console.log(path.dirname('/user/docs/file.txt'));    // /user/docs
console.log(path.extname('index.html'));                    // .html
```

✅ 5. Real-Life Uses

- To build safe file paths when reading or writing files
- To serve static files in web apps
- To get file details (like extension or file name)
- To make code run correctly on any OS (Windows, Linux, Mac)

✅ Conclusion

- The path module makes it easy to handle file and folder paths in Node.js.
- It helps in creating platform-independent paths.
- It is commonly used when working with files and directories.

4.2 Routing in Node.js

✅ 1. What is Routing?

- Routing is the process of handling different URLs (routes) in a web server.
- Based on the URL and request method (GET, POST, etc.), the server responds with different content.

🧠 Example:

- / → Home page
- /about → About page
- /contact → Contact form

✅ 2. Why Routing is Important?

- Helps in navigating different pages in a website.
- Allows creating REST APIs (e.g., /api/products, /api/users).
- Handles user requests and returns HTML, JSON, or files.

✅ 3. Basic Routing with HTTP Module

```
const http = require('http');

const server = http.createServer((req, res) => {
    if (req.url === '/') {
        res.end('Home Page');
    } else if (req.url === '/about') {
        res.end('About Page');
```

```
    } else {
        res.end('404 Page Not Found');
    }
});

server.listen(3000, () => {
    console.log('Server running on http://localhost:3000');
});
```

📌 This handles 3 routes: /, /about, and others.

✅ 4. Advanced Routing with Express (Easier)

```
const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('Home Page'));
app.get('/about', (req, res) => res.send('About Page'));

app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

📌 Express makes routing cleaner and simpler.

✅ 5. Real-Life Uses

Route	Purpose
/login	Show login form
/dashboard	Show user dashboard
/api/data	Send or receive JSON data
/products/1	Show product with ID 1

✅ Conclusion

- Routing in Node.js is used to handle different URLs and return proper responses.
- It can be done using the HTTP module or Express.js for easier handling.
- It is the core of creating web pages and APIs in Node.js.

4.3 Callback Function in Node.js

✅ 1. What is a Callback Function?

- A callback function is a function that is passed as an argument to another function.
- It is called after the main task is finished.
- Used to handle asynchronous operations (like reading a file, connecting to a server, etc.)

🧠 Think of it like:
"Do this task, and when you're done, call this function."

✅ 2. Why Are Callbacks Used in Node.js?

- Node.js is non-blocking and asynchronous.
- So instead of waiting for a task (like file reading) to finish, it continues running other code and uses a callback when the task is done.

✅ 3. Basic Example of a Callback

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

function sayBye() {
  console.log("Goodbye!");
}

greet("Rishi", sayBye);
```

📌 Output:

```
Hello Rishi
Goodbye!
```

✅ 4. Real Example in Node.js (Async)

```
const fs = require('fs');

fs.readFile('data.txt', 'utf8', function(err, data) {
  if (err) {
    console.log('Error:', err);
  } else {
    console.log('File content:', data);
  }
});
```

📌 readFile() reads the file and calls the callback after it's done.

✅ 5. Advantages of Callback Functions

Advantage	Description
🔄 Asynchronous	Don’t wait — continue other work
✅ Efficient	Uses resources better
⚙️ Flexible	Can pass any function as callback

✅ Conclusion

- A callback is a function passed to another function to be called later.
- In Node.js, callbacks are essential for async programming.
- They help keep the app non-blocking and fast.

What is Query String? How it can be parsed in Node.JS ? Give example of it.

✅ **1. What is a Query String?

- A query string is the part of a URL that comes after a ? symbol.
- It is used to pass data to the server through the URL.
- Data is sent as key-value pairs, separated by &.

🧠 Example:

http://localhost:3000/search?name=Rishi&age=22

Here:

- ? starts the query string
- name=Rishi and age=22 are key-value pairs

✅ 2. Why Use Query Strings?

- To send user input or filters in URLs
- Common in search, filters, and APIs
- Easy to access values without a form

✅ 3. Parsing Query Strings in Node.js

You can use the url and querystring modules.

✅ 4. Example: Using url and querystring modules

```
const http = require('http');
const url = require('url');
const querystring = require('querystring');

const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(req.url); // parses full URL
  const query = querystring.parse(parsedUrl.query); // parses query string

  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.write(`Hello ${query.name}, You are ${query.age} years old.`);
  res.end();
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

✅ 5. Test This in Browser

Go to:

http://localhost:3000/?name=Rishi&age=22

📌 Output:

Hello Rishi, You are 22 years old.

✅ 6. Real-Life Uses

Use Case	Example URL
Search filters	/products?category=books&price=low
User details	/profile?userId=101
Pagination	/posts?page=2&limit=10

✅ Conclusion

- A query string is used to send data through the URL.
- Node.js provides url and querystring modules to parse and extract this data.
- It’s commonly used in APIs, searches, and filters.

Unit 5: NodeJS and MongoDB

5.1 MongoDB with Node.js

1. What is MongoDB?
   - MongoDB is a NoSQL database, meaning it stores data in JSON-like format (called documents) instead of rows and columns.
   - It is schema-less, so you don't need to define the structure of your data in advance.
   - Data is stored in collections, similar to tables in SQL.

2. Why use MongoDB with Node.js?
   - MongoDB works very well with JavaScript and JSON, which fits naturally with Node.js.
   - Both are fast, scalable, and good for real-time applications like chat apps, e-commerce sites, etc.

3. Setting Up MongoDB in Node.js
   1. Install MongoDB on your system or use MongoDB Atlas (cloud version).
   2. Install MongoDB driver or Mongoose in your project:

```
npm install mongoose
```

4. Connecting to MongoDB using Mongoose

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydb', {
    useNewUrlParser: true,
    useUnifiedTopology: true
}).then(() => {
    console.log("MongoDB Connected");
}).catch(err => {
    console.log(err);
});
```

5. Creating a Schema and Model

```
const userSchema = new mongoose.Schema({
    name: String,
    age: Number,
    email: String
});

const User = mongoose.model('User', userSchema);
```

6. CRUD Operations

✓ Create:

```
const user = new User({ name: "Rishi", age: 22, email: "rishi@example.com" });
user.save();
```

✅ Read:

```
User.find().then(users => console.log(users));
```

✅ Update:

```
User.updateOne({ name: "Rishi" }, { age: 23 });
```

✅ Delete:

```
User.deleteOne({ name: "Rishi" });
```

7. Advantages of Using MongoDB in Node.js

- Flexible data model.
- High performance.
- Easy integration with Node.js.
- Scalable for big applications.

Conclusion

MongoDB is a powerful NoSQL database that fits perfectly with Node.js projects. With the help of libraries like Mongoose, we can easily perform database operations, making full-stack development easier and faster.

5.2 MongoDB Objects in Node.js

1. What are MongoDB Objects?

- In MongoDB (with Node.js), data is stored as documents, which are basically JavaScript objects.
- These objects follow a BSON format (Binary JSON), which supports additional data types like ObjectId, Date, etc.

2. Common MongoDB Object Types in Node.js

MongoDB Type	JavaScript Equivalent	Example
ObjectId	mongoose.Types.ObjectId	_id: ObjectId("...")
String	String	"name": "Rishi"
Number	Number	"age": 22
Boolean	Boolean	"isActive": true
Date	Date object	"createdAt": new Date()
Array	Array	"hobbies": ["music", "gym"]
Object	Object	"address": { city: "Surat" }

3. Example MongoDB Object (Document)

```
{
  _id: ObjectId("6617467af2"),
  name: "Rishi",
  age: 22,
```

```
    email: "rishi@example.com",
    isActive: true,
    hobbies: ["coding", "editing"],
    address: { city: "Surat", pincode: 395007 },
    createdAt: new Date()
}
```

4. Working with MongoDB Objects in Node.js using Mongoose

Step 1: Define a Schema

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
    name: String,
    age: Number,
    email: String,
    isActive: Boolean,
    hobbies: [String],
    address: {
        city: String,
        pincode: Number
    },
    createdAt: { type: Date, default: Date.now }
});
```

Step 2: Create a Model

```
const User = mongoose.model('User', userSchema);
```

Step 3: Create MongoDB Object (Document)

```
const user = new User({
    name: "Rishi",
    age: 22,
    email: "rishi@example.com",
    isActive: true,
    hobbies: ["coding", "editing"],
    address: { city: "Surat", pincode: 395007 }
});

user.save(); // Save object to database
```

5. ObjectId – Special MongoDB Object

- Every document in MongoDB has a unique _id of type ObjectId.
- You can create it manually:

```
const { ObjectId } = require('mongodb');
const id = new ObjectId();
console.log(id); // Unique ID
```

6. Accessing Object Properties

```
User.find().then(users => {
   users.forEach(user => {
      console.log(user.name); // Accessing object property
   });
});
```

Conclusion

MongoDB Objects in Node.js are simply JavaScript-like structures that hold data. With tools like Mongoose, we can define, create, and use these objects easily to interact with MongoDB in a structured and efficient way.

Working with MongoDB – Writing Data in Node.js

✅ 1. What is Writing Data?

- Writing data means inserting or saving information into a MongoDB database.
- Example: adding a new user, product, or message.

✅ 2. Tools Required

- Node.js – Server-side platform.
- MongoDB – NoSQL database.
- Mongoose – Library to connect MongoDB with Node.js.

🔻 Install Mongoose:

```
npm install mongoose
```

✅ 3. Connect Node.js with MongoDB

```
const mongoose = require('mongoose');

mongoose.connect("mongodb://localhost:27017/mydb", {
   useNewUrlParser: true,
   useUnifiedTopology: true
}).then(() => {
   console.log("MongoDB Connected");
}).catch(err => {
   console.log("Connection Error:", err);
});
```

✅ 4. Create a Schema (Structure of Data)

```
const userSchema = new mongoose.Schema({
   name: String,
   age: Number,
```

```
    email: String
});
```

✅ 5. Create a Model (Like a Class)

```
const User = mongoose.model("User", userSchema);
```

✅ 6. Insert/Write One Document (Data Entry)

```
const user = new User({
    name: "Rishi",
    age: 22,
    email: "rishi@example.com"
});

user.save().then(() => {
    console.log("User saved!");
});
```

✅ 7. Insert Many Documents

```
User.insertMany([
    { name: "Amit", age: 25, email: "amit@example.com" },
    { name: "Priya", age: 23, email: "priya@example.com" }
]).then(() => {
    console.log("Multiple users saved!");
});
```

✅ 8. Output Example

```
MongoDB Connected
User saved!
Multiple users saved!
```

Working with MongoDB – Querying in Node.js

✅ 1. What is Querying?

- Querying means searching or finding data from a MongoDB database.
- Example: Find all users above age 18 or find a user by email.

✅ 2. Tools Used

- Node.js – Backend environment
- MongoDB – NoSQL database
- Mongoose – Helps us query data easily in Node.js

🔹 Install Mongoose:

```
npm install mongoose
```

✅ 3. Connect Node.js with MongoDB

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydb', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log("Connected to MongoDB");
});
```

✅ 4. Create Schema and Model

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
  email: String
});

const User = mongoose.model('User', userSchema);
```

✅ 5. Query Examples

🔹 a) Find All Documents

```
User.find().then(users => {
  console.log(users);
});
```

🔹 b) Find by Name

```
User.find({ name: "Rishi" }).then(result => {
  console.log(result);
});
```

🔹 c) Find One Record

```
User.findOne({ email: "rishi@example.com" }).then(user => {
  console.log(user);
});
```

🔹 d) Find by ID

```
User.findById("661123abc456def789").then(user => {
  console.log(user);
});
```

✅ 6. Use of Query Operators

Operator	Meaning	Example
$gt	Greater than	{ age: { $gt: 20 } }
$lt	Less than	{ age: { $lt: 30 } }

$in	Value in array	{ name: { $in: ["Rishi", "Amit"] } }
$and	Combine conditions	{ $and: [{ age: 22 }, { name: "Rishi" }] }

✅ 7. Sorting and Limiting Results

🔹 Sort by Age:

```
User.find().sort({ age: 1 }); // 1 for ascending, -1 for descending
```

🔹 Limit Results:

```
User.find().limit(3); // Only first 3 records
```

✅ 8. Output Example

```
[
  { name: "Rishi", age: 22, email: "rishi@example.com" },
  { name: "Amit", age: 25, email: "amit@example.com" }
]
```

✅ 9. Conclusion

- Querying in MongoDB with Node.js is easy using Mongoose.
- We can find, filter, sort, and limit data from our database using simple functions.
- This is very useful for making dashboards, search features, user lists, etc.

Working with MongoDB – Indexes in Node.js

✅ 1. What are Indexes in MongoDB?

- Indexes are used to speed up searching in a MongoDB database.
- Without indexes, MongoDB must check every document — this is slow when there's lots of data.
- Think of an index in a book — it helps you find topics faster.

✅ 2. Why Use Indexes?

- Faster queries on large data
- Improves performance of apps
- Reduces time for find(), sort(), etc.

✅ 3. Default Index

MongoDB automatically creates an index on the _id field of every document.

```
// _id index is created by default
{ _id: 1 }
```

✅ 4. Creating Indexes in Node.js (Mongoose)

🔹 Create Index on Email

```
User.collection.createIndex({ email: 1 });
```

1 = ascending order

-1 = descending order

✅ 5. Creating Compound Index (Multiple Fields)

```
User.collection.createIndex({ name: 1, age: -1 });
```

Speeds up queries that filter by name and age

✅ 6. Using Indexes in Schema

You can also define an index in your Mongoose schema:

```
const userSchema = new mongoose.Schema({
  name: String,
  email: { type: String, index: true },    // Index added here
  age: Number
});
```

✅ 7. Query with Indexed Field

```
User.find({ email: "rishi@example.com" }) // This will be faster if 'email' is indexed
```

✅ 8. View All Indexes

```
User.collection.indexes().then(indexes => {
  console.log(indexes);
});
```

✅ 9. Remove an Index

```
User.collection.dropIndex("email_1"); // Index name is usually fieldname_order
```

✅ 10. Conclusion

- Indexes in MongoDB make searching faster and improve performance.
- Use them on fields that are frequently searched, sorted, or filtered (like email, username, date).
- In Node.js, use Mongoose or native MongoDB commands to create and manage indexes easily.

Working with MongoDB – MapReduce in Node.js

✅ 1. What is MapReduce?

- MapReduce is a powerful tool in MongoDB used to:
  - Process large amounts of data
  - Group, filter, or summarize data (like counting or totaling)
- It uses Map and Reduce functions to transform and combine data.

✅ 2. Why Use MapReduce?

When you want to perform operations like:

- Counting how many users are of each age
- Total sales by category
- Average ratings by movie

✅ 3. Components of MapReduce

Part	Meaning
Map	Extracts key-value pairs from each document
Reduce	Combines values with the same key
Out	Where to store the final results

✅ 4. Example Collection (users)

```
[
  { "name": "Rishi", "age": 22 },
  { "name": "Amit", "age": 22 },
  { "name": "Priya", "age": 23 }
]
```

✅ 5. MapReduce Example in Node.js

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydb', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

const userSchema = new mongoose.Schema({
  name: String,
  age: Number
});

const User = mongoose.model("User", userSchema);

// Perform MapReduce
User.mapReduce({
  map: function () {
    emit(this.age, 1); // Group by age
  },
  reduce: function (key, values) {
    return Array.sum(values); // Count users with same age
  },
  out: { inline: 1 } // Show result in output, not in a collection
}).then(results => {
  console.log("MapReduce Results:");
  console.log(results);
```

```
});
```

✅ 6. Output of the Above Code

```
[
  { _id: 22, value: 2 },
  { _id: 23, value: 1 }
]
```

🟢 This means:

Age 22 → 2 users

Age 23 → 1 user

✅ 7. Store Results in a New Collection

out: { replace: "age_summary" }

🔽 This creates a new collection age_summary with results.

✅ 8. Use Cases of MapReduce

- Count users by city
- Total sales per month
- Average ratings by movie
- Group orders by product

✅ 9. When Not to Use MapReduce

- If data is small, use aggregation instead (simpler and faster).
- MapReduce is best for large datasets and custom logic.

✅ 10. Conclusion

- MapReduce in MongoDB with Node.js helps to process and summarize large data.
- It uses map to pick data and reduce to combine results.
- In Node.js, we use Mongoose’s mapReduce() method to run it easily.

Node.js - Express Framework

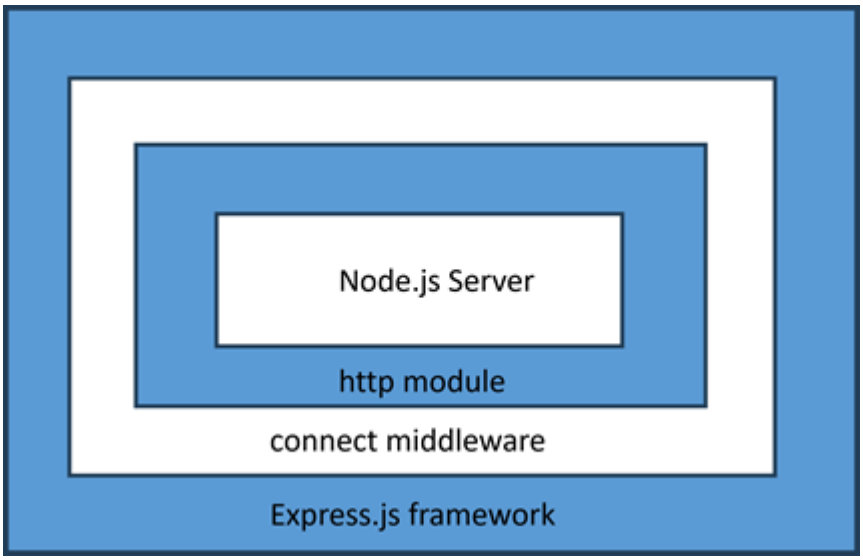
Express.js is a minimal and flexible web application framework that provides a robust set of features to develop Node.js based web and mobile applications. Express.js is one of the most popular web frameworks in the Node.js ecosystem. Express.js provides all the features of a modern web framework, such as templating, static file handling, connectivity with SQL and NoSQL databases.

Node.js has a built-in web server. The createServer() method in its http module launches an asynchronous http server. It is possible to develop a web application with core Node.js features. However, all the low level manipulations of HTTP request and responses have to be tediously handled. The web application frameworks take care of these common tasks, allowing the developer to concentrate on the business logic of the application. A web framework such as Express.js is a set of utilities that facilitates rapid, robust and scalable web applications.

Following are some of the core features of Express framework –

- Allows to set up middlewares to respond to HTTP Requests.
- Defines a routing table which is used to perform different actions based on HTTP Method and URL.
- Allows to dynamically render HTML Pages based on passing arguments to templates.

The Express.js is built on top of the connect middleware, which in turn is based on http, one of the core modules of Node.js API.



Installing Express

The Express.js package is available on npm package repository. Let us install express package locally in an application folder named ExpressApp.

```
D:\expressApp> npm init
D:\expressApp> npm install express --save
```

The above command saves the installation locally in the node_modules directory and creates a directory express inside node_modules.

Hello world Example

Following is a very basic Express app which starts a server and listens on port 5000 for connection. This app responds with Hello World! for requests to the homepage. For every other path, it will respond with a 404 Not Found.

```
var express = require('express');
var app = express();

app.get('/', function (req, res) {
   res.send('Hello World');
})

var server = app.listen(5000, function () {
   console.log("Express App running at http://127.0.0.1:5000/");
})
```

Save the above code as index.js and run it from the command-line.

```
D:\expressApp> node index.js
Express App running at http://127.0.0.1:5000/
```

Visit http://localhost:5000/ in a browser window. It displays the Hello World message.

Application object

An object of the top level express class denotes the application object. It is instantiated by the following statement –

```
var express = require('express');
var app = express();
```

The Application object handles important tasks such as handling HTTP requests, rendering HTML views, and configuring middleware etc.

The app.listen() method creates the Node.js web server at the specified host and port. It encapsulates the createServer() method in http module of Node.js API.

```
app.listen(port, callback);
```

Basic Routing

The app object handles HTTP requests GET, POST, PUT and DELETE with app.get(), app.post(), app.put() and app.delete() method respectively. The HTTP request and HTTP response objects are provided as arguments to these methods by the NodeJS server. The first parameter to these methods is a string that represents the endpoint of the URL.

These methods are asynchronous, and invoke a callback by passing the request and response objects.

GET method

In the above example, we have provided a method that handles the GET request when the client visits '/' endpoint.

```
app.get('/', function (req, res) {
   res.send('Hello World');
})
```

- Request Object – The request object represents the HTTP request and has properties for the request query string, parameters, body, HTTP headers, and so on.
- Response Object – The response object represents the HTTP response that an Express app sends when it gets an HTTP request. The send() method of the response object formulates the server's response to the client.

You can print request and response objects which provide a lot of information related to HTTP request and response including cookies, sessions, URL, etc.

The response object also has a sendFile() method that sends the contents of a given file as the response.

```
res.sendFile(path)
```

Save the following HTML script as index.html in the root folder of the express app.

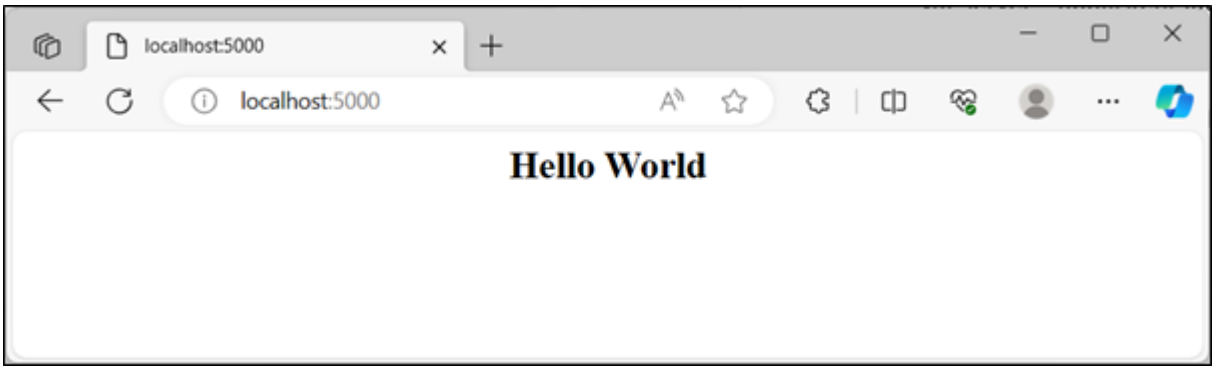
```
<html>
<body>
<h2 style="text-align: center;">Hello World</h2>
</body>
</html>
```

Change the index.js file to the code below –

```
var express = require('express');
var app = express();
var path = require('path');
```

```
app.get('/', function (req, res) {
   res.sendFile(path.join(__dirname,"index.html"));
})

var server = app.listen(5000, function () {

   console.log("Express App running at http://127.0.0.1:5000/");
})
```

Run the above program and visit http://localhost:5000/, the browser shows Hello World message as below:



Let us use sendFile() method to display a HTML form in the index.html file.

```
<html>
   <body>

      <form action = "/process_get" method = "GET">
         First Name: <input type = "text" name = "first_name">  <br>
         Last Name: <input type = "text" name = "last_name">  <br>
         <input type = "submit" value = "Submit">
      </form>

   </body>
</html>
```

The above form submits the data to /process_get endpoint, with GET method. Hence we need to provide a app.get() method that gets called when the form is submitted.

```
app.get('/process_get', function (req, res) {
   // Prepare output in JSON format
   response = {
      first_name:req.query.first_name,
```

```
      last_name:req.query.last_name
   };
   console.log(response);
   res.end(JSON.stringify(response));
})
```

The form data is included in the request object. This method retrieves the data from request.query array, and sends it as a response to the client.

The complete code for index.js is as follows –

```
var express = require('express');
var app = express();
var path = require('path');

app.use(express.static('public'));

app.get('/', function (req, res) {
   res.sendFile(path.join(__dirname,"index.html"));
})

app.get('/process_get', function (req, res) {
   // Prepare output in JSON format
   response = {
      first_name:req.query.first_name,
      last_name:req.query.last_name
   };
   console.log(response);
   res.end(JSON.stringify(response));
})

var server = app.listen(5000, function () {
   console.log("Express App running at http://127.0.0.1:5000/");
})
```

Visit http://localhost:5000/.

Now you can enter the First and Last Name and then click submit button to see the result and it should return the following result −

```
{"first_name":"John","last_name":"Paul"}
```

POST method

The HTML form is normally used to submit the data to the server, with its method parameter set to POST, especially when some binary data such as images is to be submitted. So, let us change the method parameter in index.html to POST, and action parameter to "process_POST".

```
<html>
   <body>

      <form action = "/process_POST" method = "POST">
         First Name: <input type = "text" name = "first_name">  <br>
         Last Name: <input type = "text" name = "last_name">  <br>
         <input type = "submit" value = "Submit">
      </form>

   </body>
</html>
```

To handle the POST data, we need to install the body-parser package from npm. Use the following command.

```
npm install body-parser save
```

This is a node.js middleware for handling JSON, Raw, Text and URL encoded form data.

This package is included in the JavaScript code with the following require statement.

```
var bodyParser = require('body-parser');
```

The urlencoded() function creates application/x-www-form-urlencoded parser

```
// Create application/x-www-form-urlencoded parser
var urlencodedParser = bodyParser.urlencoded({ extended: false })
```

Add the following app.post() method in the express application code to handle POST data.

```
app.post('/process_post', urlencodedParser, function (req, res) {
   // Prepare output in JSON format
   response = {
      first_name:req.body.first_name,
      last_name:req.body.last_name
   };
   console.log(response);
   res.end(JSON.stringify(response));
})
```

Here is the complete code for index.js file

```
var express = require('express');
var app = express();
var path = require('path');

var bodyParser = require('body-parser');
// Create application/x-www-form-urlencoded parser
var urlencodedParser = bodyParser.urlencoded({ extended: false })

app.use(express.static('public'));

app.get('/', function (req, res) {
   res.sendFile(path.join(__dirname,"index.html"));
})

app.get('/process_get', function (req, res) {
   // Prepare output in JSON format
   response = {
      first_name:req.query.first_name,
```

```
      last_name:req.query.last_name
   };
   console.log(response);
   res.end(JSON.stringify(response));
})
app.post("/process_post", )
var server = app.listen(5000, function () {
   console.log("Express App running at http://127.0.0.1:5000/");
})
```

Run index.js from command prompt and visit http://localhost:5000/.

Now you can enter the First and Last Name and then click the submit button to see the following result –

```
{"first_name":"John","last_name":"Paul"}
```

Serving Static Files

Express provides a built-in middleware express.static to serve static files, such as images, CSS, JavaScript, etc.

You simply need to pass the name of the directory where you keep your static assets, to the express.static middleware to start serving the files directly. For example, if you keep your images, CSS, and JavaScript files in a directory named public, you can do this –

```
app.use(express.static('public'));
```

We will keep a few images in public/images sub-directory as follows –

```
node_modules
index.js
public/
```

```
public/images
public/images/logo.png
```

Let's modify "Hello Word" app to add the functionality to handle static files.

```
var express = require('express');
var app = express();
app.use(express.static('public'));

app.get('/', function (req, res) {
   res.send('Hello World');
})

var server = app.listen(5000, function () {
   console.log("Express App running at http://127.0.0.1:5000/");
})
```

Save the above code in a file named index.js and run it with the following command.

```
D:\expressApp> node index.js
```

Now open http://127.0.0.1:5000/images/logo.png in any browser and see observe following result.

To learn Express.js in details, visit our ExpressJS Tutorial (ExpressJS)

📄 Express Supportive Modules: body-parser and method-override

These are middleware modules that help Express handle form data and HTTP method limitations easily.

1️⃣ **body-parser Module**

🔷 What is it?
body-parser is a middleware used to parse (understand) the incoming request body (data sent from forms, JSON, etc.) and make it available in req.body.

Without it, Express can't read form or JSON data directly.

🔧 How to install:
npm install body-parser

🧠 Why use it?
To read data from POST, PUT, or PATCH requests.
Works with HTML forms or JSON requests.

✅ Example:

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();

// Middleware
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

app.post('/submit', (req, res) => {
  const name = req.body.name;
  res.send(`Hello, ${name}`);
});

// Start server
app.listen(3000, () => {
  console.log('Server started');
});
```

📌 HTML Form (used with it):

```
<form method="POST" action="/submit">
  <input type="text" name="name" />
  <button type="submit">Submit</button>
</form>
```

➡️ When submitted, the server can read the form data from req.body.name.

2️⃣ method-override Module

🔷 What is it?
HTML forms only support GET and POST methods.
But in real web apps (REST APIs), we need PUT, DELETE, etc.

method-override allows you to simulate PUT or DELETE using a hidden form field or query string.

🔧 How to install:

```
npm install method-override
```

🧠 Why use it?
To perform PUT or DELETE operations using HTML forms.
Helps when building RESTful apps (like update or delete a product).

✅ Example:

```
const express = require('express');
const methodOverride = require('method-override');
const app = express();

// Enable form data parsing
app.use(express.urlencoded({ extended: true }));

// Override with _method query or hidden input
app.use(methodOverride('_method'));

app.delete('/delete-user', (req, res) => {
  res.send('User deleted');
});

// Start server
app.listen(3000, () => {
  console.log('Server running');
});
```

📌 HTML Form (used with method-override):

```
<form method="POST" action="/delete-user?_method=DELETE">
  <button type="submit">Delete User</button>
</form>
```

Even though the form sends POST, method-override will convert it to DELETE for the route.

✨ Summary Table:

Module	Purpose	Example Use Case

body-parser	Parse form or JSON data into req.body	Reading submitted form values
method-override	Simulate PUT or DELETE in HTML forms	REST APIs like /update, /delete routes

🔚 Final Thoughts:
body-parser helps Express apps read user input
method-override helps simulate full HTTP methods for RESTful APIs

Use EJS as Template Engine in Node.js

Last Updated : 16 Jan, 2025

EJS (Embedded JavaScript) is a popular templating engine for Node.js that allows you to generate HTML markup with plain JavaScript. It is particularly useful for creating dynamic web pages, as it enables you to embed JavaScript logic directly within your HTML.

EJS

EJS or Embedded Javascript Templating is a templating engine used by Node.js. Template engine helps to create an HTML template with minimal code. Also, it can inject data into an HTML template on the client side and produce the final HTML. EJS is a simple templating language that is used to generate HTML markup with plain JavaScript. It also helps to embed JavaScript into HTML pages.

Steps to Use EJS as Template Engine

Installing EJS

To begin with, using EJS as templating engine we need to install EJS using the given command:

```
npm install express ejs --save
```

It will install exxpress and ejs as dependency in the node.js project.

Set Up Your Node Application

```
// Filename - index.js

// Set express as Node.js web application
// server framework.

const express = require('express');
const app = express();

// Set EJS as templating engine
app.set('view engine', 'ejs');
```

Create the EJS Template

The default behavior of EJS is that it looks into the 'views' folder for the templates to render. So, let's make a 'views' folder in our main node project folder and make a file named "home.ejs" which is to be served on some desired requests in our node project. The content of this page is:

```
<!-- Home.ejs -->

<!DOCTYPE html>
<html>
<head>
    <title>Home Page</title>

    <style type="text/css" media="screen">
        body {
            background-color: skyblue;
            text-decoration-color: white;
            font-size: 7em;
        }
    </style>
</head>

<body>
    <center>This is our home page.</center>
</body>
</html>
```

Now, we will render this page on a certain request by the user:

```
app.get('/', (req, res) => {

    // The render method takes the name of the HTML
    // page to be rendered as input
    // This page should be in the views folder
    // in the root directory.
    res.render('home');
```

```
});
```

Now, the page home.ejs will be displayed on requesting localhost. To add dynamic content this render method takes a second parameter which is an object. This is done as:

```
app.get('/', (req, res) => {

    // The render method takes the name of the HTML
    // page to be rendered as input.
    // This page should be in views folder in
    // the root directory.
    // We can pass multiple properties and values
    // as an object, here we are passing the only name
    res.render('home', { name: 'Akashdeep' });
});

const server = app.listen(4000, function () {
    console.log('listening to port 4000')
});
```

Now, We will embed the name to the HTML page as:

```
<!DOCTYPE html>
<html>
<head>
    <title>Home Page</title>
    <style type="text/css" media="screen">
        body {
            background-color: skyblue;
            text-decoration-color: white;
            font-size: 7em;
        }
    </style>
</head>

<body>
    <center>
        This is our home page.<br />
        Welcome <%=name%>, to our home page.
    </center>
</body>
</html>
```

It is used to embed dynamic content to the page and is used to embed normal JavaScript. Now embedding normal JavaScript:

```
app.get('/', (req, res) => {

    // The render method takes the name of the html
```

```
    // page to be rendered as input.
    // This page should be in views folder
    // in the root directory.
    let data = {
        name: 'Akashdeep',
        hobbies: ['playing football', 'playing chess', 'cycling']
    }

    res.render('home', { data: data });
});

const server = app.listen(4000, function () {
    console.log('listening to port 4000')
});
```

The final HTML content:

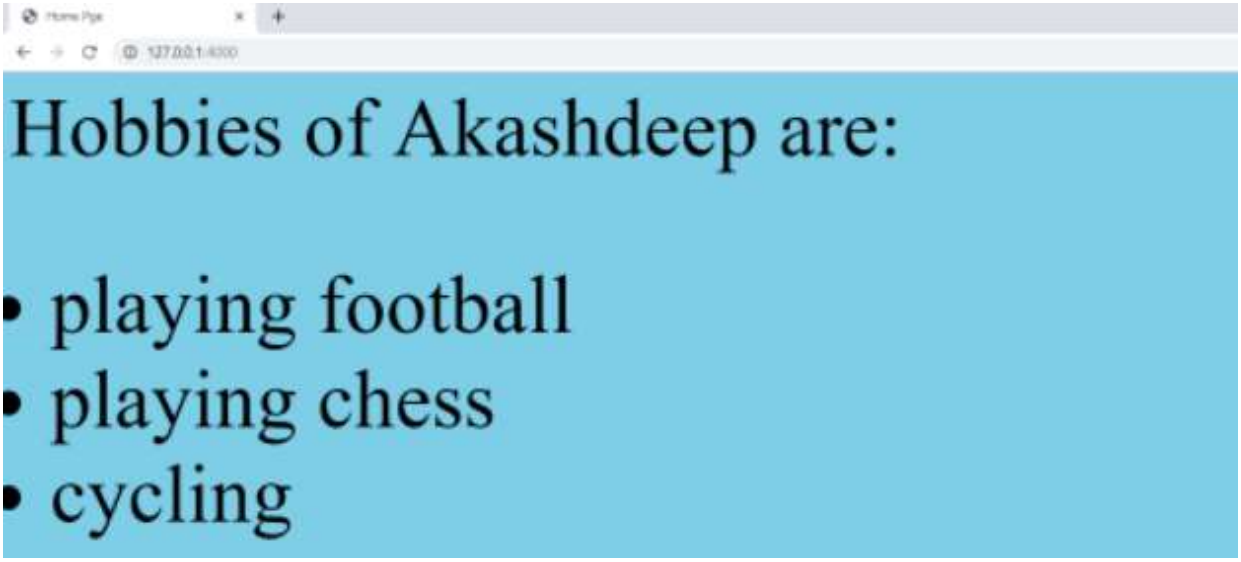
```
<!-- Home.ejs -->

<!DOCTYPE html>
<html>
<head>
    <title>Home Page</title>
    <style type="text/css" media="screen">
        body {
            background-color: skyblue;
            text-decoration-color: white;
            font-size: 5em;
        }
    </style>
</head>

<body>
    Hobbies of <%=data.name%> are:<br />

        <ul>
            <% data.hobbies.forEach((item)=>{%>
                <li>
                    <%=item%>
                </li>
                <%});%>
        </ul>
</body>
</html>
```

Steps to run the program: After creating all the files go to the root directory of your project folder and run the below command

```
node index.js
```

Type the node file_name.js command to run your program and see the output as displayed.

Output:



Key Features of EJS

- **Embedded JavaScript:** Allows you to embed JavaScript logic directly within your HTML.
- **Partial Templates:** Supports partials, enabling you to reuse common template fragments (like headers and footers) across different pages.
- **Layout Support:** EJS can be used with layout managers to create consistent layouts across multiple views.

Conclusion

EJS is a powerful and flexible templating engine that enhances your Node.js applications by allowing you to generate dynamic HTML content with ease. Its simplicity and integration with Express make it an ideal choice for developers looking to build server-rendered web applications quickly. Whether you're developing a simple site or a complex web application, EJS provides the tools you need to create dynamic and interactive user interfaces.

Node.js - MongoDB Get Started

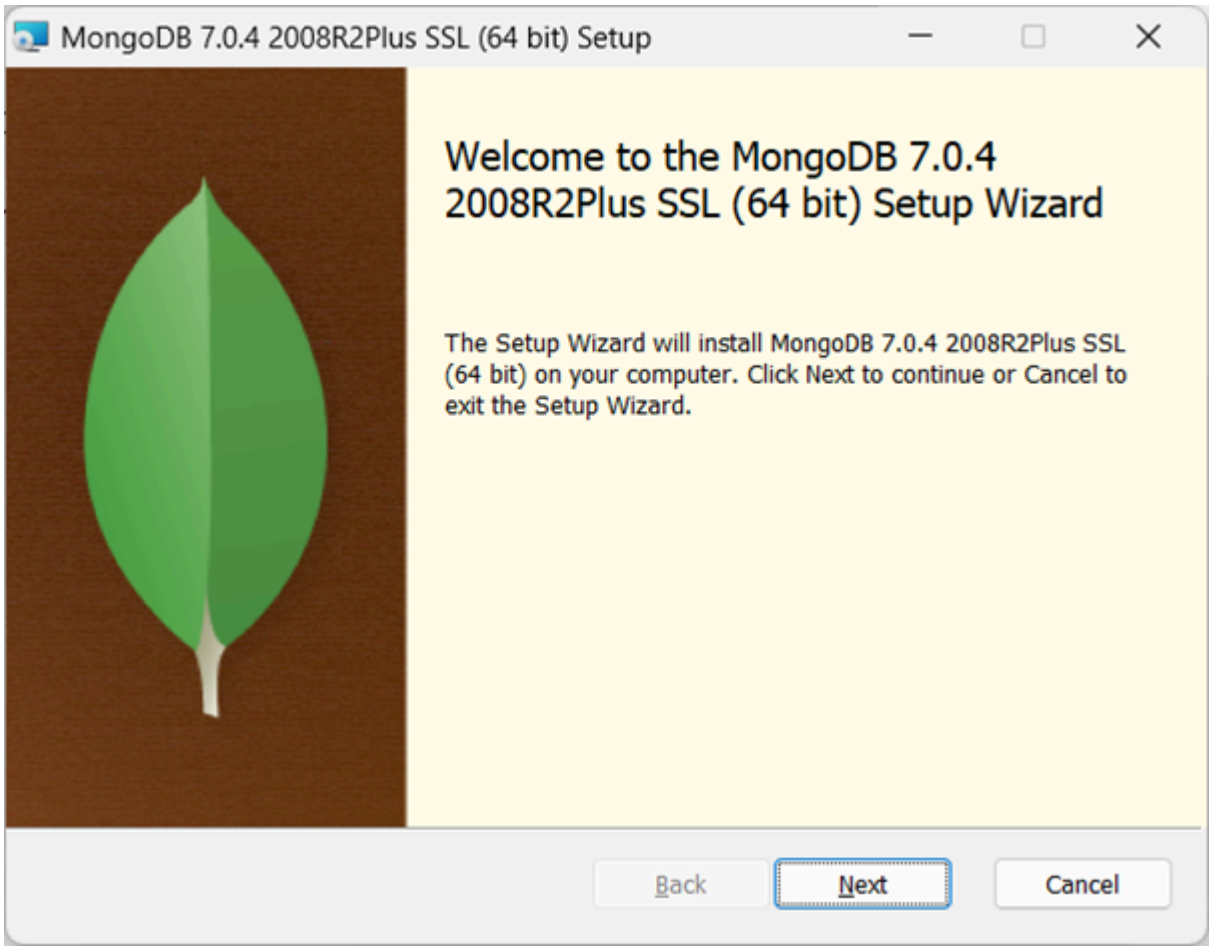
In this chapter, we shall learn to use MongoDB as a backend with Node.js application. A Node.js application can be interfaced with MongoDB through the NPM module called mongodb itself. MongoDB is a document-oriented NOSQL database. It is an open-source, distributed and horizontally scalable schema-free database architecture. Since MongoDB internally uses JSON-like format (called BSON) for data storage and transfer, it is a natural companion of Node.js, itself a JavaScript runtime, used for server-side processing.

Installation

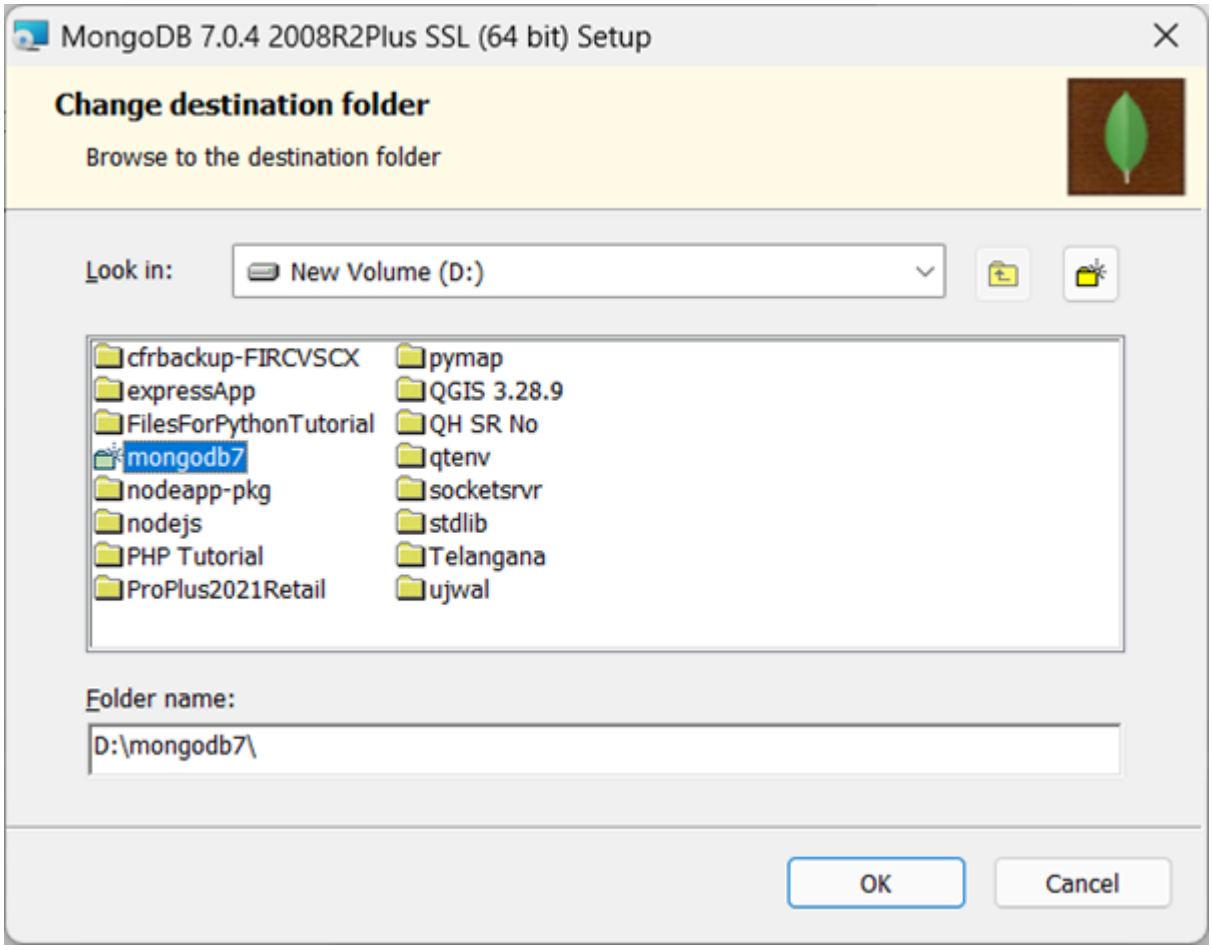
MongoDB server software is available in two forms: Community edition and Enterprise edition. The MongoDB community edition is available for Windows, Linux as well as MacOS operating systems at https://www.mongodb.com/try/download/community

Windows

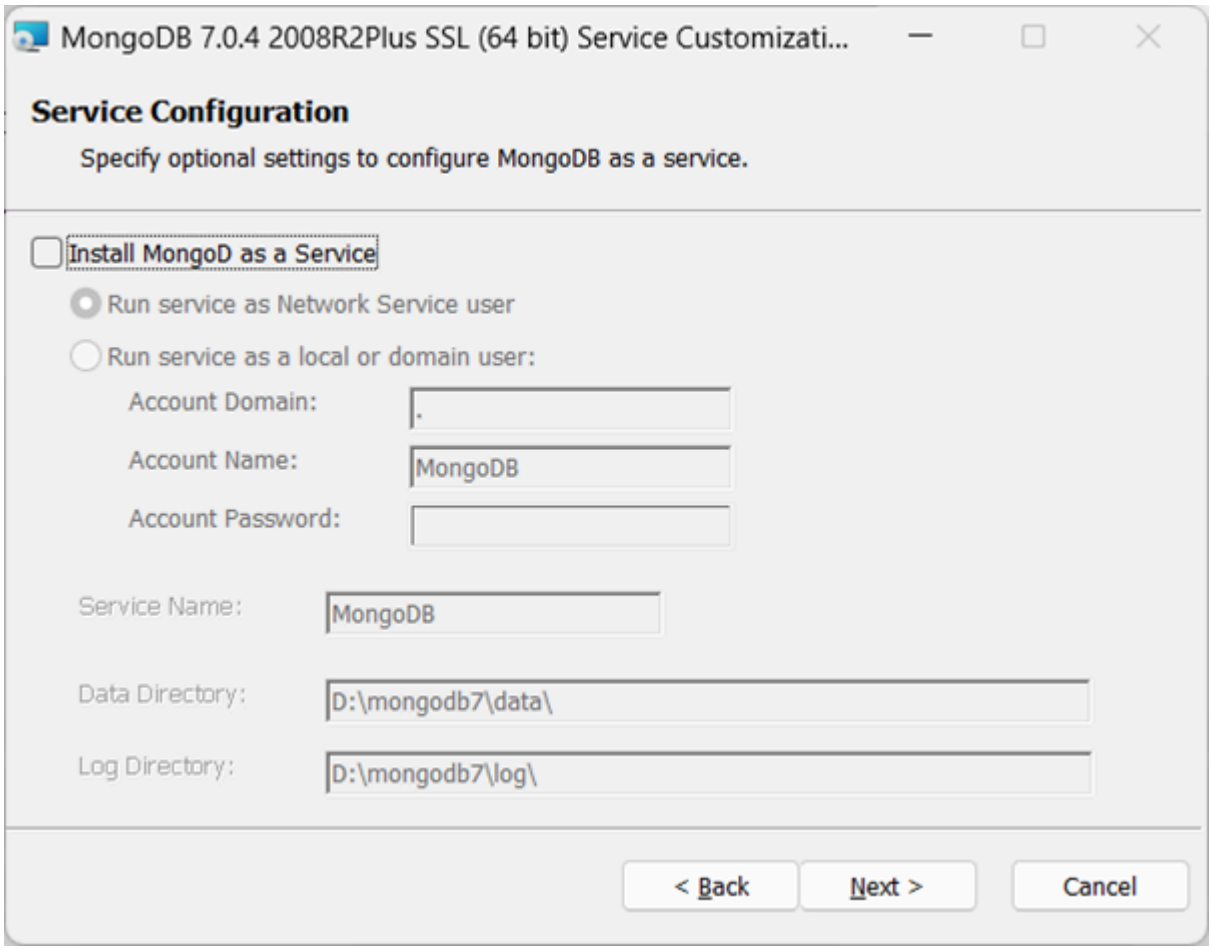
Download the latest version of MongoDB installer (https://fastdl.mongodb.org/windows/mongodb-windows-x86_64-7.0.4-signed.msi) and start the installation wizard by double-clicking the file.



Choose Custom installation option to specify a convenient installation folder.



Uncheck the "Install MongoD as service" option, and accept the default data and log directory options.

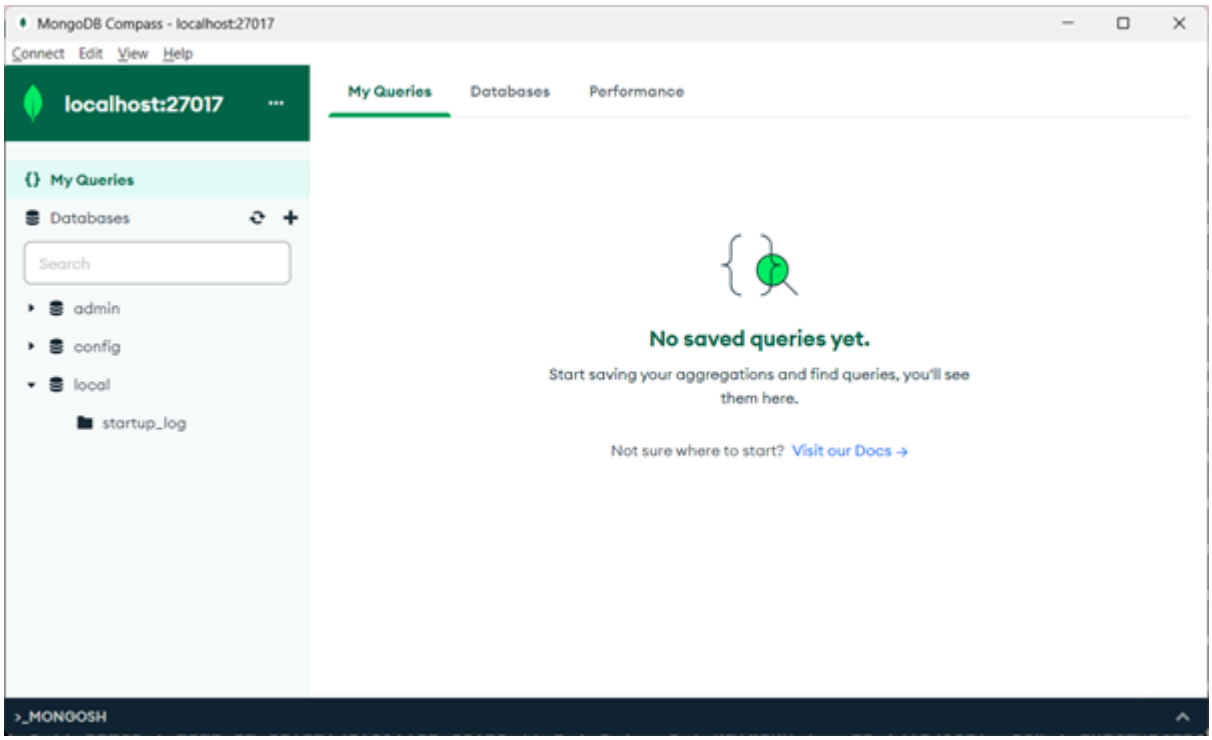


Go through the rest of steps in the installation wizard.

Create a directory "d:\data\db" and specify it as dbpath in the following command-line to start the MongoDB server –

```
"D:\mongodb7\mongod.exe" --dbpath="d:\data\db"
```

The installer also guides you to install MongoDB Compass, a GUI client for interacting with the MongoDB server. MongoDB server starts listening to the incoming connection requests at the 27017 port by default. Start the Compass app, and connect to the server using the default connection string as 'mongodb://localhost:27017'.



Ubuntu

To install MongoDB server on Ubuntu Linux, issue the following command to reload the local package database −

```
sudo apt-get update
```

You can now install either the latest stable version of MongoDB or a specific version of MongoDB.

To install the latest stable version, issue the following

```
sudo apt-get install -y mongodb-org
```

mongodb driver

Now we need to install the mongodb driver module from the NPM repository, so that a Node.js application could interact with MongoDB.

Initialize a new Node.js application in a new folder with the following command −

```
D:\nodejs\mongoproj>npm init -y
Wrote to D:\nodejs\mongoproj\package.json:

{
   "name": "mongoproj",
```

```
   "version": "1.0.0",
   "description": "",
   "main": "index.js",
   "scripts": {
      "test": "echo \"Error: no test specified\" && exit 1"
   },
   "keywords": [],
   "author": "",
   "license": "ISC"
}
```

Install the mongodb driver module from NPM repository with the following command –

```
D:\nodejs\mongoproj>npm install mongodb
```

Connecting to MongoDB

Now we can establish connection with MongoDB server. First, import the MongoClient class from the mongodb module with the require() statement. Call its connect() method by passing the MongoDB server URL.

```
const { MongoClient } = require('mongodb');

// Connection URL
const url = 'mongodb://localhost:27017';
const client = new MongoClient(url);

// Database Name
const dbName = 'myProject';

async function main() {
   // Use connect method to connect to the server
   await client.connect();
   console.log('Connected successfully to server');
   const db = client.db(dbName);
   const collection = db.collection('documents');

   // the following code examples can be pasted here...

   return 'done.';
}
```

```
main()
.then(console.log)
.catch(console.error)
.finally(() => client.close());
```

Assuming that the above script is saved as app.js, run the application from command prompt –

```
PS D:\nodejs\mongoproj> node app.js
Connected successfully to server
done.
```

Node.js - RESTful API

A Node.js application using ExpressJS is ideally suited for building REST APIs. In this chapter, we shall explain what is a REST (also called RESTFul) API, and build a Node.js based, Express.js REST application. We shall also use REST clients to test out REST API.

API is an acronym for Application Programming Interface. The word interface generally refers to a common meeting ground between two isolated and independent environments. A programming interface is an interface between two software applications. The term REST API or RESTFul API is used for a web Application that exposes its resources to other web/mobile applications through the Internet, by defining one or more endpoints which the client apps can visit to perform read/write operations on the host's resources.

REST architecture has become the de facto standard for building APIs, preferred by the developers over other technologies such as RPC, (stands for Remote Procedure Call), and SOAP (stands for Simple Object Access Protocol).

What is REST architecture?

REST stands for REpresentational State Transfer. REST is a well known software architectural style. It defines how the architecture of a web application should behave. It is a resource based architecture where everything that the REST server hosts, (a file, an image, or a row in a table of a database), is a resource, having many representations. REST was first introduced by Roy Fielding in 2000.

REST recommends certain architectural constraints.

- Uniform interface
- Statelessness
- Client-server
- Cacheability

- Layered system
- Code on demand

These are the advantages of REST constraints –

- Scalability
- Simplicity
- Modifiability
- Reliability
- Portability
- Visibility

A REST Server provides access to resources and REST client accesses and modifies the resources using HTTP protocol. Here each resource is identified by URIs/ global IDs. REST uses various representation to represent a resource like text, JSON, XML but JSON is the most popular one.

HTTP methods

Following four HTTP methods are commonly used in REST based architecture.

POST Method

The POST verb in the HTTP request indicates that a new resource is to be created on the server. It corresponds to the CREATE operation in the CRUD (CREATE, RETRIEVE, UPDATE and DELETE) term. To create a new resource, you need certain data, it is included in the request as a data header.

Examples of POST request –

```
HTTP POST http://example.com/users
HTTP POST http://example.com/users/123
```

GET Method

The purpose of the GET operation is to retrieve an existing resource on the server and return its XML/JSON representation as the response. It corresponds to the READ part in the CRUD term.

Examples of a GET request –

```
HTTP GET http://example.com/users
HTTP GET http://example.com/users/123
```

PUT Method

The client uses HTTP PUT method to update an existing resource, corresponding to the UPDATE part in CRUD). The data required for update is included in the request body.

Examples of a PUT request –

```
HTTP PUT http://example.com/users/123
HTTP PUT http://example.com/users/123/name/Ravi
```

DELETE Method

The DELETE method (as the name suggest) is used to delete one or more resources on the server. On successful execution, an HTTP response code 200 (OK) is sent.

Examples of a DELETE request –

```
HTTP DELETE http://example.com/users/123
HTTP DELETE http://example.com/users/123/name/Ravi
```

RESTful Web Services

Web services based on REST Architecture are known as RESTful web services. These webservices uses HTTP methods to implement the concept of REST architecture. A RESTful web service usually defines a URI, Uniform Resource Identifier a service, which provides resource representation such as JSON and set of HTTP Methods.

Creating RESTful API for A Library

Consider we have a JSON based database of users having the following users in a file users.json:

```
{
   "user1" : {
      "name" : "mahesh",
      "password" : "password1",
      "profession" : "teacher",
```

```
      "id": 1
   },

   "user2" : {
      "name" : "suresh",
      "password" : "password2",
      "profession" : "librarian",
      "id": 2
   },

   "user3" : {
      "name" : "ramesh",
      "password" : "password3",
      "profession" : "clerk",
      "id": 3
   }
}
```

Our API will expose the following endpoints for the clients to perform CRUD operations on the users.json file, which the collection of resources on the server.

Sr.No.	URI	HTTP Method	POST body	Result
1	/	GET	empty	Show list of all the users.
2	/	POST	JSON String	Add details of new user.
3	/:id	DELETE	JSON String	Delete an existing user.
4	/:id	GET	empty	Show details of a user.
5	/:id	PUT	JSON String	Update an existing user

List Users

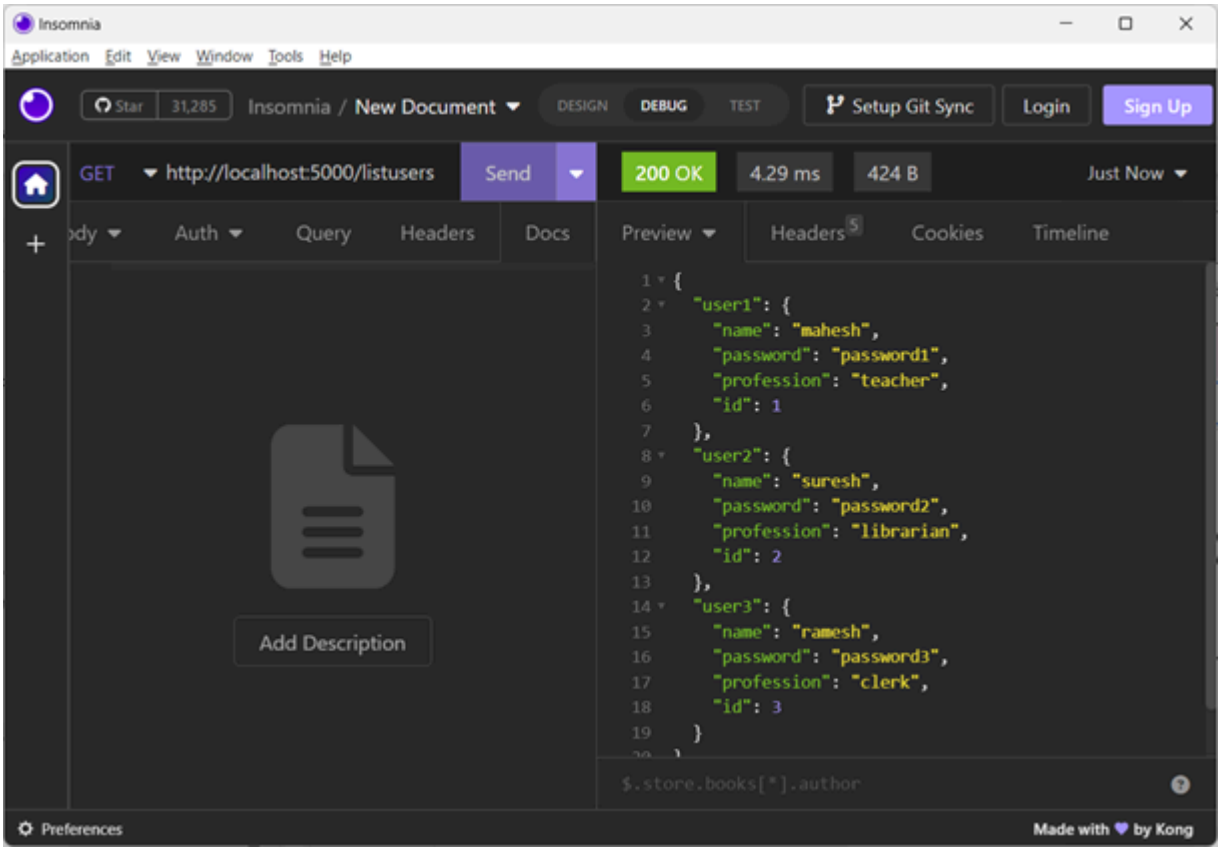
Let's implement the first route in our RESTful API to list all Users using the following code in a index.js file

```
var express = require('express');
var app = express();
var fs = require("fs");
app.get('/', function (req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {
```

```
      res.end( data );
   });
})
var server = app.listen(5000, function () {
   console.log("Express App running at http://127.0.0.1:5000/");
})
```

To test this endpoint, you can use a REST client such as Postman or Insomnia. In this chapter, we shall use Insomnia client.

Run index.js from command prompt, and launch the Insomnia client. Choose GET methos and enter http://localhost:5000/ URL. The list of all users from users.json will be displayed in the Respone Panel on right.



You can also use CuRL command line tool for sending HTTP requests. Open another terminal and issue a GET request for the above URL.

```
C:\Users\mlath>curl http://localhost:5000/
{
   "user1" : {
      "name" : "mahesh",
      "password" : "password1",
      "profession" : "teacher",
      "id": 1
   },
```

```
   "user2" : {
      "name" : "suresh",
      "password" : "password2",
      "profession" : "librarian",
      "id": 2
   },

   "user3" : {
      "name" : "ramesh",
      "password" : "password3",
      "profession" : "clerk",
      "id": 3
   }
}
```

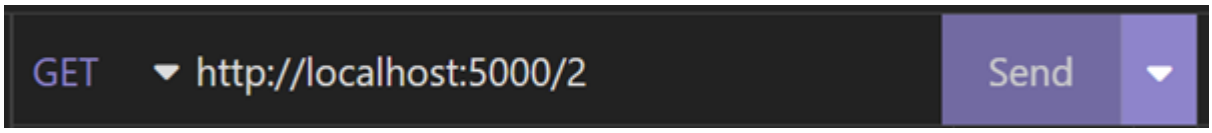
Show Detail

Now we will implement an API endpoint /:id which will be called using user ID and it will display the detail of the corresponding user.

Add the following method in index.js file −

```
app.get('/:id', function (req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {
      var users = JSON.parse( data );
      var user = users["user" + req.params.id]
      res.end( JSON.stringify(user));
   });
})
```

In the Insomnia interface, enter http://localhost:5000/2 and send the request.



You may also use the CuRL command as follows to display the details of user2 −

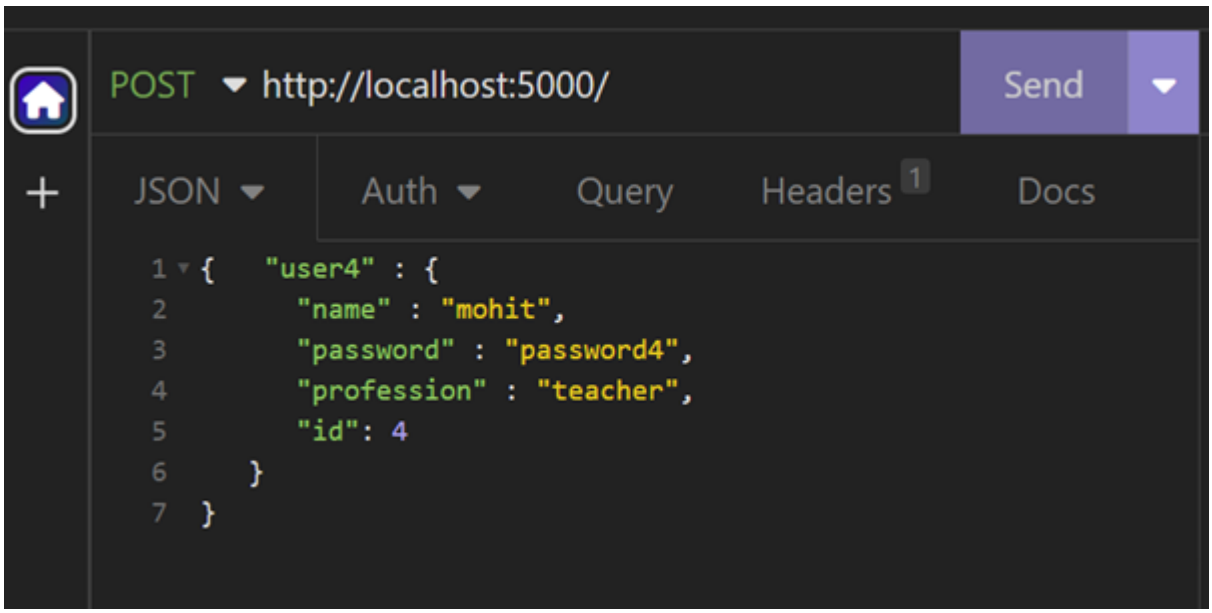
```
C:\Users\mlath>curl http://localhost:5000/2
{"name":"suresh","password":"password2","profession":"librarian","id":2}
```

Add User

Following API will show you how to add new user in the list. Following is the detail of the new user. As explained earlier, you must have installed body-parser package in your application folder.

```
var bodyParser = require('body-parser')
app.use( bodyParser.json() );
app.use(bodyParser.urlencoded({  extended: true }));

app.post('/', function (req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {
      var users = JSON.parse( data );
      var user = req.body.user4;
      users["user"+user.id] = user
      res.end( JSON.stringify(users));
   });
})
```

To send POST request through Insomnia, set the BODY tab to JSON, and enter the the user data in JSON format as shown



You will get a JSON data of four users (three read from the file, and one added)

```
{
    "user1": {
        "name": "mahesh",
        "password": "password1",
        "profession": "teacher",
```

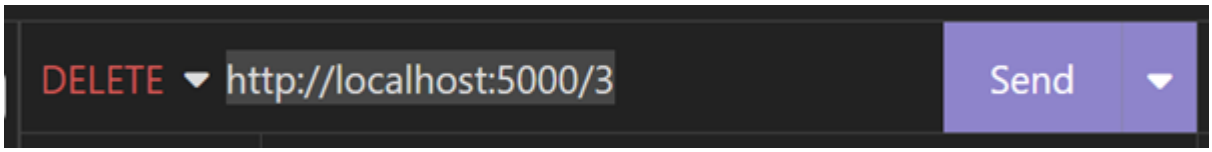
```
        "id": 1
    },
    "user2": {
        "name": "suresh",
        "password": "password2",
        "profession": "librarian",
        "id": 2
    },
    "user3": {
        "name": "ramesh",
        "password": "password3",
        "profession": "clerk",
        "id": 3
    },
    "user4": {
        "name": "mohit",
        "password": "password4",
        "profession": "teacher",
    "id": 4
    }
  }
```

Delete user

The following function reads the ID parameter from the URL, locates the user from the list that is obtained by reading the users.json file, and the corresponding user is deleted.

```
app.delete('/:id', function (req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {
      data = JSON.parse( data );
      var id = "user"+req.params.id;
      var user = data[id];
      delete data[ "user"+req.params.id];
      res.end( JSON.stringify(data));
   });
})
```

Choose DELETE request in Insomnia, enter http://localhost:5000/2 and send the request. The user with ID=3 will be deleted, the remaining users are listed in the response panel



Output

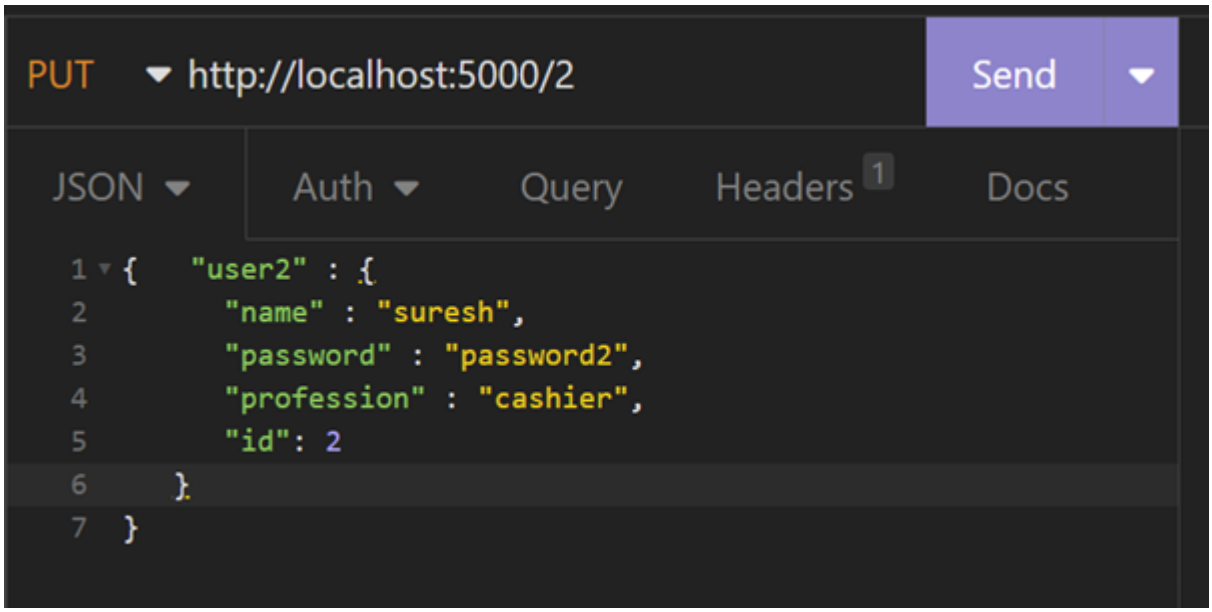
```
{
    "user1": {
        "name": "mahesh",
        "password": "password1",
        "profession": "teacher",
        "id": 1
    },
    "user2": {
        "name": "suresh",
        "password": "password2",
        "profession": "librarian",
        "id": 2
    }
}
```

Update user

The PUT method modifies an existing resource with the server. The following app.put() method reads the ID of the user to be updated from the URL, and the new data from the JSON body.

```
app.put("/:id", function(req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {

      var users = JSON.parse( data );
      var id = "user"+req.params.id;
      users[id]=req.body;
      res.end( JSON.stringify(users));
   })
})
```

In Insomnia, set the PUT method for http://localhost:5000/2 URL.



The response shows the updated details of user with ID=2

```
{
    "user1": {
        "name": "mahesh",
        "password": "password1",
        "profession": "teacher",
    "id": 1
    },
    "user2": {
        "name": "suresh",
        "password": "password2",
        "profession": "Cashier",
        "id": 2
    },
    "user3": {
        "name": "ramesh",
        "password": "password3",
        "profession": "clerk",
        "id": 3
    }
}
```

Here is the complete code for the Node.js RESTFul API −

```
var express = require('express');
var app = express();
var fs = require("fs");
```

```
var bodyParser = require('body-parser')
app.use( bodyParser.json() );
app.use(bodyParser.urlencoded({  extended: true }));


app.get('/', function (req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {
      res.end( data );
   });
})

app.get('/:id', function (req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {
      var users = JSON.parse( data );
      var user = users["user" + req.params.id]
     res.end( JSON.stringify(user));
   });
})

var bodyParser = require('body-parser')
app.use( bodyParser.json() );
app.use(bodyParser.urlencoded({  extended: true }));

app.post('/', function (req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {
      var users = JSON.parse( data );
      var user = req.body.user4;
      users["user"+user.id] = user
      res.end( JSON.stringify(users));
   });
})

app.delete('/:id', function (req, res) {
   fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err, data) {
      data = JSON.parse( data );
      var id = "user"+req.params.id;
      var user = data[id];
      delete data[ "user"+req.params.id];
      res.end( JSON.stringify(data));
   });
})
app.put("/:id", function(req, res) {
```

```
      fs.readFile( __dirname + "/" + "users.json", 'utf8', function (err,
data) {

      var users = JSON.parse( data );
      var id = "user"+req.params.id;

      users[id]=req.body;
      res.end( JSON.stringify(users));
   })

})
var server = app.listen(5000, function () {
   console.log("Express App running at http://127.0.0.1:5000/");
})
```

Node.js - Events

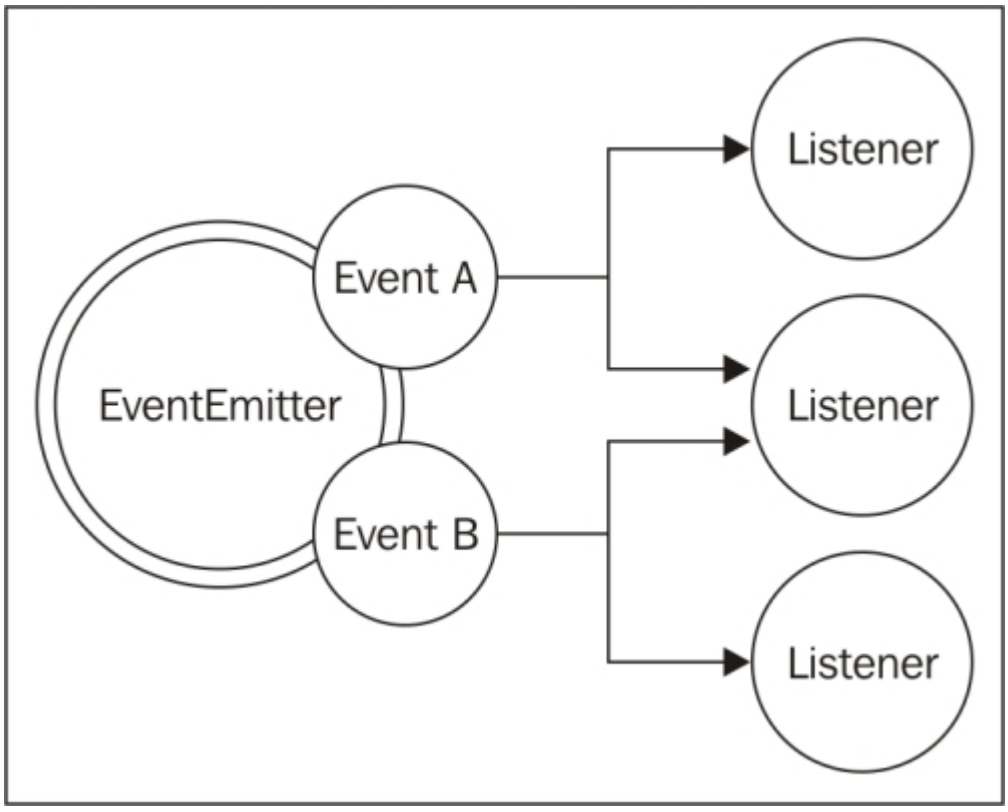
When JavaScript is used inside HTML script, it generally handles the user-generated events such as button press or mouse clicks. The core API of Node.js is an asynchronous event-driven architecture. However, unlike the client-side JavaScript, it handles the events on the server, such as File io operations, server's request and responses etc.

The net.Server object emits an event each time a peer connects to it.

The ReadStream referring to a file emits an event when the file is opened and whenever data is available to be read.

Node.js identifies several types of events. Each event can be attached to a callback function. Whenever an event occurs, the callback attached to it is triggered. The Node.js runtime is always listening to events that may occur. When any event that it can identify occurs, its attached callback function is executed.

The Node.js API includes events module, consisting mainly the EventEmitter class. An EventEmmiter object triggers (or emits) a certain type of event. You can assign one or more callbacks (listeners) to a certain type of event. whenever that event triggers, all the registered callbacks are fired one by one in order to which they were registered.



These are the steps involved in event handling in Node.js API.

First, import the events module, and declare an object of EventEmitter class

```
// Import events module
var events = require('events');

// Create an eventEmitter object
var eventEmitter = new events.EventEmitter();
```

Bind an event handler with an event with the following syntax –

```
// Bind event and event  handler as follows
eventEmitter.on('eventName', eventHandler);
```

To fire the event programmatically –

```
// Fire an event
eventEmitter.emit('eventName');
```

Example

Given below is a simple example that binds two listeners to the on event of EventEmitter class

</> Open Compiler

```
// Import events module
var events = require('events');

// Create an eventEmitter object
var eventEmitter = new events.EventEmitter();

// Create an event handler as follows
var connectHandler = function connected() {
   console.log('connection successful.');
}

// Bind the connection event with the handler
eventEmitter.on('connection', connectHandler);

// Bind the data_received event with the anonymous function
eventEmitter.on('data_received', function(){
   console.log('data received successfully.');
});

// Fire the connection event
eventEmitter.emit('connection');

// Fire the data_received event
eventEmitter.emit('data_received');
console.log("Program Ended.");
```

Output

```
connection successful.
data received successfully.
Program Ended.
```

Any asynchronous function in Node Application accepts a callback as the last parameter. The callback function accepts an error as the first parameter.

Create a text file named input.txt with the following content.

```
Tutorials Point is giving self learning content
to teach the world in simple and easy way!!!!!
```

Create a js file named main.js having the following code –

```
var fs = require("fs");
fs.readFile('input.txt', function (err, data) {
   if (err) {
      console.log(err.stack);
      return;
   }
   console.log(data.toString());
});
console.log("Program Ended");
```

Here fs.readFile() is a async function that read a file. If an error occurs during the read operation, then the err object will contain the corresponding error, else data will contain the contents of the file. readFile passes err and data to the callback function after the read operation is complete, which finally prints the content.

```
Program Ended
Tutorials Point is giving self learning content
to teach the world in simple and easy way!!!!!
```

Node.js - Event Emitter

The Node.js API is based on an event-driven architecture. It includes the events module, which provides the capability to create and handle custom events. The event module contains EventEmitter class. The EventEmitter object emits named events. Such events call the listener functions. The Event Emitters have a very crucial role the Node.js ecosystem. Many objects in a Node emit events, for example, a net.Server object emits an event each time a peer connects to it, or a connection is closed. The fs.readStream object emits an event when the file is opened, closed, a read/write operation is performed. All objects which emit events are the instances of events.EventEmitter.

Since the EvenEmitter class is defined in events module, we must include in the code with require statement.

```
var events = require('events');
```

To emit an event, we should declare an object of EventEmitter class.

```
var eventEmitter = new events.EventEmitter();
```

When an EventEmitter instance faces any error, it emits an 'error' event. When a new listener is added, 'newListener' event is fired and when a listener is removed, 'removeListener' event is fired.

Sr.No.	Events & Description
1	**newListener(event, listener)** ■ **event** – String: the event name ■ **listener** – Function: the event handler function

	This event is emitted any time a listener is added. When this event is triggered, the listener may not yet have been added to the array of listeners for the event.
2	**removeListener(event, listener)** ■ **event** − String The event name ■ **listener** − Function The event handler function This event is emitted any time someone removes a listener. When this event is triggered, the listener may not yet have been removed from the array of listeners for the event.

Following instance methods are defined in EventEmitter class −

Sr.No.	Events & Description
1	**addListener(event, listener)** Adds a listener at the end of the listeners array for the specified event. Returns emitter, so calls can be chained.
2	**on(event, listener)** Adds a listener at the end of the listeners array for the specified event. Same as addListener.
3	**once(event, listener)** Adds a one time listener to the event. This listener is invoked only the next time the event is fired, after which it is removed
4	**removeListener(event, listener)** Removes a listener from the listener array for the specified event. If any single listener has been added multiple times to the listener array for the specified event, then removeListener must be called multiple times to remove each instance.
5	**removeAllListeners([event])** Removes all listeners, or those of the specified event. It's not a good idea to remove listeners that were added elsewhere in the code, especially when it's on an emitter that you didn't create (e.g. sockets or file streams).
6	**setMaxListeners(n)** By default, EventEmitters will print a warning if more than 10 listeners are added for a particular event. Set to zero for unlimited.
7	**listeners(event)** Returns an array of listeners for the specified event.

8	**emit(event, [arg1], [arg2], [...])** Execute each of the listeners in order with the supplied arguments. Returns true if the event had listeners, false otherwise.
9	**off(event, listener)** Alias for removeListener

Example

Let us define two listener functions as below –

```
var events = require('events');
var eventEmitter = new events.EventEmitter();

// listener #1
var listner1 = function listner1() {
   console.log('listner1 executed.');
}

// listener #2
var listner2 = function listner2() {
   console.log('listner2 executed.');
}
```

Let us bind these listeners to a connection event. The first function listener1 is registered with addListener() method, while we use on() method to bind listener2.

```
// Bind the connection event with the listner1 function
eventEmitter.addListener('connection', listner1);

// Bind the connection event with the listner2 function
eventEmitter.on('connection', listner2);

// Fire the connection event
eventEmitter.emit('connection');
```

When the connection event is fired with emit() method, the console shows the log message in the listeners as above

```
listner1 executed.
```

```
listner2 executed.
```

Let us remove the listener2 callback from the connection event, and fire the connection event again.

```
// Remove the binding of listner1 function
eventEmitter.removeListener('connection', listner1);
console.log("Listner1 will not listen now.");

// Fire the connection event
eventEmitter.emit('connection');
```

The console now shows the following log −

```
listner1 executed.
listner2 executed.
Listner1 will not listen now.
listner2 executed.
```

The EventEmitter class also has a listCount() method. It is a class method, that returns the number of listeners for a given event.

</> Open Compiler

```
const events = require('events');

const myEmitter = new events.EventEmitter();
// listener #1
var listner1 = function listner1() {
   console.log('listner1 executed.');
}

// listener #2
var listner2 = function listner2() {
  console.log('listner2 executed.');
}

// Bind the connection event with the listner1 function
myEmitter.addListener('connection', listner1);
```

```
// Bind the connection event with the listner2 function
myEmitter.on('connection', listner2);

// Fire the connection event
myEmitter.emit('connection');
console.log("Number of Listeners:" + myEmitter.listenerCount('connection'));
```

Output

```
listner1 executed.
listner2 executed.
Number of Listeners:2
```

📃 Event Listener in Node.js

🔷 Definition:

- An Event Listener in Node.js is a function that waits (or listens) for a specific event to happen and then executes a block of code when that event occurs.
- In Node.js, we use the EventEmitter class (from the built-in events module) to create and handle events.
- We use .on() or .once() methods to add event listeners.

🔁 How It Works:

Node.js follows an event-driven architecture, which means:

- Instead of running code line-by-line and waiting,
- It listens for events and responds only when the event happens.

Example events:

- A file is read
- A button is clicked
- A user logs in

✅ How to Create an Event Listener:

```
// Step 1: Import the events module
const EventEmitter = require('events');

// Step 2: Create an emitter object
const emitter = new EventEmitter();

// Step 3: Add a listener for a custom event
emitter.on('greet', () => {
    console.log('Hello! This is an event listener in action.');
});

// Step 4: Emit (trigger) the event
emitter.emit('greet');
```

📌 Output:

Hello! This is an event listener in action.

🧠 Explanation:

'greet' is the event name.
emitter.on('greet', function) adds a listener that waits for the 'greet' event.
emitter.emit('greet') triggers the event, and the listener executes the handler function.

✅ Key Features of Event Listeners:

Feature	Description
.on('event', fn)	Listens for the event every time it occurs.

.once('event', fn)	Listens for the event only once — then removes itself.
Multiple Listeners	You can add more than one listener for the same event.
Arguments Supported	Listeners can receive data passed during emit()

🔄 Real-Life Analogy:
🔔 Event = Someone rings the doorbell
👂 Event Listener = You're listening for the bell
🚪 Event Handler = You open the door when you hear it

🌟 Why Event Listeners are Important:

- Used in file reading, server requests, real-time apps
- Helps Node.js stay non-blocking and asynchronous
- Makes code modular and easy to maintain
- Very useful in chat apps, notifications, gaming apps, etc.

📃 Event Handler in Node.js

🔷 Definition:
An Event Handler in Node.js is the function that gets executed when a specific event is triggered.
👉 In simple words:

- The event listener waits for an event.
- When the event happens, the event handler runs the code.
  🔁 Listener = Waiter 👂
  👉 Handler = Action 🚀

✅ Example:

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Event Handler Function
function handleLogin() {
   console.log('User has logged in.');
}

// Add listener and handler
emitter.on('login', handleLogin);

// Trigger the event
emitter.emit('login');
```

📌 Output:
User has logged in.

🔍 Explanation:

- emitter.on('login', handleLogin) → This attaches the handler to the login event.
- handleLogin → This is the event handler function. It runs when the event is emitted.
- emitter.emit('login') → This triggers the event, causing the handler to execute.

🧠 Key Points:

Term	Meaning
Event	Something that happens (e.g., 'login', 'dataReady')
Event Listener	Watches for that event (.on('login', ...))
Event Handler	The actual code/function that runs when the event happens

🛎️ Real-Life Analogy:

Event: Someone rings the doorbell 🔔

Listener: You're listening for the bell 👂

Handler: You open the door 🚪

So the handler is the reaction to an event!

🧩 Methods to Attach Handlers:

Method	Description
.on()	Attaches a handler that runs every time
.once()	Handler runs only once and then removes
.removeListener()	Removes a specific handler
.removeAllListeners()	Removes all handlers for an event

🌟 Why Event Handlers are Important in Node.js:

- Handle user interactions like login, logout, clicks
- React to system events like file read, request received
- Helps make real-time and non-blocking applications
- Makes code modular, clean, and maintainable

Non-Blocking event loop in Node.js

Last Updated : 18 Jun, 2024

Node.js operates on a single-threaded, event-driven architecture that relies heavily on non-blocking I/O operations to handle concurrent requests efficiently. This approach is enabled by its event loop mechanism, which allows Node.js to handle multiple requests concurrently without creating additional threads for each request. Let's explore how the non-blocking event loop works in Node.js.

Think of a waiter working in a restaurant. He goes around the restaurant taking orders from customers and serving them when their respective food is ready. What happens when the waiter takes an order from a customer and waits until the food is prepared, serves it and then proceeds to the next customer. This way the waiting time for each customer increases and the restaurant would be a huge flop. The latter represents synchronous programming and the earlier one represents asynchronous programming.

Event-Driven Architecture

Node.js uses an event-driven, asynchronous model to handle requests and I/O operations. Here's how it typically works:

- **Event Loop**: Node.js includes an event loop that continuously listens for events and executes callback functions when an event occurs. It's the heart of Node.js architecture, managing asynchronous operations.
- **Non-Blocking I/O**: Node.js performs I/O operations asynchronously, meaning it doesn't wait for these operations to complete before moving on to the next task. Instead, it delegates I/O tasks to the

operating system and registers callback functions to be executed once the operation completes.
- **Callback Queue**: When an asynchronous operation completes, its callback function is pushed into the callback queue.
- **Event-Driven Execution**: The event loop picks up callback functions from the callback queue and executes them one by one in the order they were queued, when the stack is empty.

Benefits of Non-Blocking Event Loop

- **Scalability**: Node.js can handle large numbers of concurrent connections efficiently because it doesn't create threads for each connection. Instead, it uses a single-threaded event loop to manage all connections asynchronously.
- **Performance**: By leveraging non-blocking I/O, Node.js can perform multiple tasks concurrently without being blocked by slow I/O operations. This improves overall application performance and responsiveness.
- **Resource Efficiency**: Node.js consumes less memory compared to traditional multi-threaded servers because it doesn't allocate resources for each individual thread. Instead, it manages concurrent requests with a single thread and an event loop.

Non-Blocking

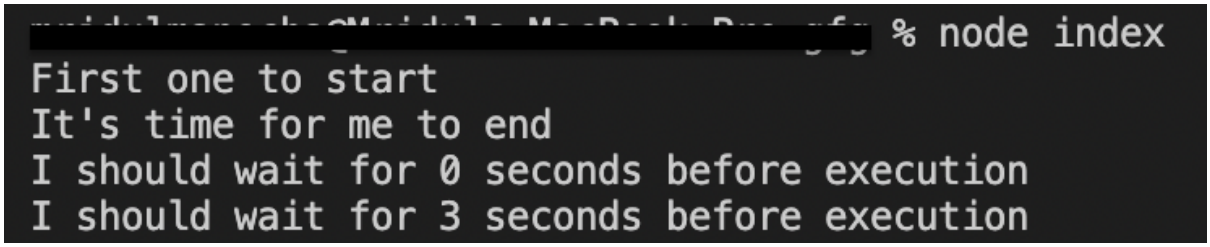
Non-Blocking nature of node.js simply means that node.js proceeds with the execution of the program instead of waiting for long I/O operations or HTTP requests. i.e the non-JavaScript related code is processed in the background by different threads or by the browser which gets executed when node.js completes execution of the main program and when the data required is successfully fetched. This way, the long time taking operations do not block the execution of the remaining part of the program. Hence the name Non-Blocking.

Example: Have a look at the below code-snippet for getting a better understanding of the non-blocking nature of Node.js.

```
console.log("First one to start");
```

```
setTimeout(() => {
    console.log("I should wait for 3 seconds before execution");
}, 3000);

setTimeout(() => {
    console.log("I should wait for 0 seconds before execution");
}, 0);

console.log("It's time for me to end");
```

Output:



As you can see, the rest of the program is unaffected by the long delay. But why did “I should wait for 0 seconds before execution” print after “It’s time for me to end”. Well, let’s get deeper into the internal working of Node.js.

How the code is Executed

Step 1: The main function is pushed to the call stack.

Step 2: The first console.log statement is pushed to the call stack.

Step 3: “First one to start” is printed and the console.log function is removed from the call stack.

Step 4: The setTimeOut(3000) function is given to the browser for processing.

Step 5: The setTimeOut(0) function is given to the browser for processing.

Step 6: The setTimeOut(0) function has been processed and given to the CallBack Queue.

Step 7: The last console.log statement is pushed to the call stack.

Step 8: “It’s time for me to end” is printed and the console.log function is removed from the call stack.

Step 9: The main function is removed from the call stack and the event loop starts running.

Step 10: The console.log function present in setTimeOut(0) is pushed to the call stack.

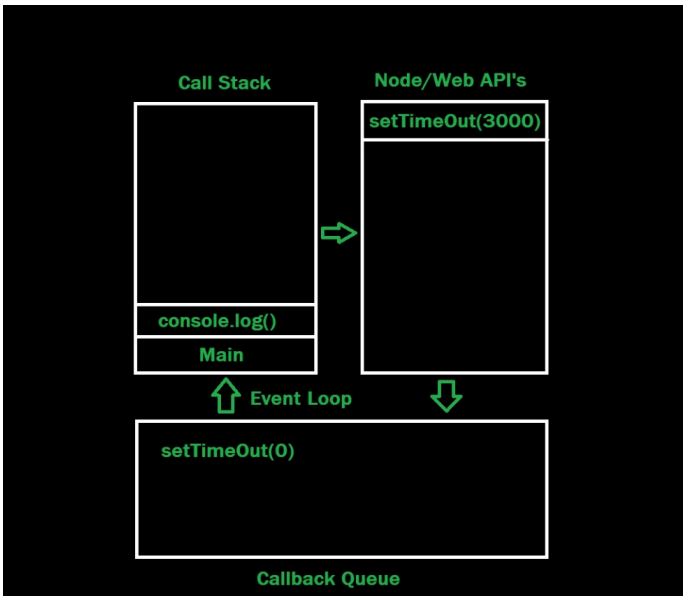
Step 11: “I should wait for 0 seconds before execution” is printed and the console.log function is removed from the call stack.

Step 12: After waiting for 3 seconds, the browser gives the setTimeOut(3000) function to the CallBack Queue.

Step 13: The console.log function present in setTimeOut(3000) is pushed to the call stack.

Step 14: “I should wait for 3 seconds before execution” is printed and the console.log function is removed from the call stack.

Diagram:



Non-Blocking Event Loop

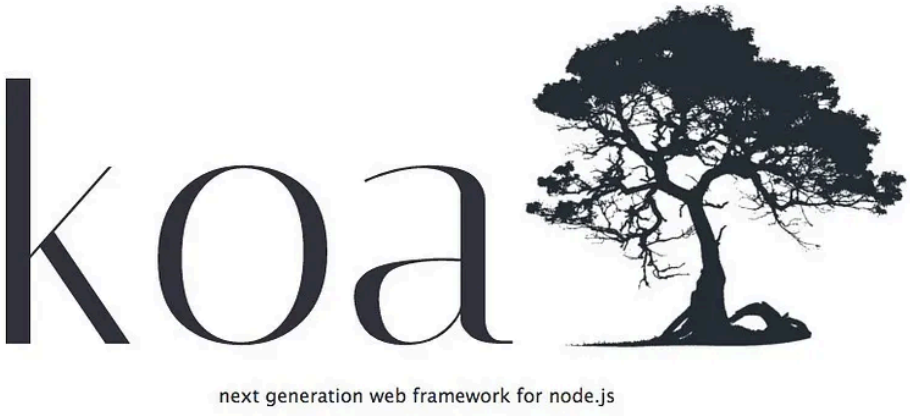
Conclusion

Node.js’ non-blocking event loop and asynchronous I/O model enable it to efficiently handle high concurrency and I/O-bound operations. By leveraging a single-threaded event loop and non-blocking operations, Node.js achieves high performance and scalability, making it well-suited for building fast and responsive applications, particularly those requiring real-time interactions or handling numerous concurrent connections. Understanding and leveraging these principles is crucial for maximizing the benefits of Node.js in modern web development.

Next-generation web framework for node.js (Koa.js)

MadhushaPrasad · Follow

6 min read · Aug 10, 2023



The Express development team created Koa.js, an open-source Node.js web framework. As their official website says, the framework aims to be a smaller, more expressive, and more robust foundation for web applications and APIs.Koa considerably enhances error handling and helps do away with callbacks through the use of asynchronous functions. Koa's core does not include any middleware. It offers a sophisticated set of techniques that expedite and enhance the server-making process.

Who uses Koa.js?

The Koa.js framework is widely used by businesses for their websites and online APIs. Here is a list of the top five Koa.js framework users from across the world.

1. Paralect

2. Pubu
3. Bulb
4. GAPO
5. Clovis

Features of Koa.js

Koa.js has its own special features that make it more expressive and developer-friendly, much like all other accessible frameworks. Here are a couple of Koa's best features.

1. **It is modern and futuristic.**

Koa.js is modern and future-proof Unlike other Node.js frameworks, Koa.js build is based on ES6 which makes the development of complex applications simpler by providing a bunch of new classes and modules. This helps developers in creating maintainable applications.

2. It has a small footprint.

Koa.js is less in weight than other Node.js frameworks. This aids in the creation of middleware that is lighter and more effective. However, they do have the choice to add additional modules to the system to make it larger to accommodate the demands of varied projects.

3. It uses ES6 generators

Koa.js leverages ES6 generators to make synchronous programming easier and to improve control flow. In order to manage the execution of code on the same stack, these generators can also be utilized as functions.

4. **Better Error handling**

Koa.js enhances error handling by leveraging middleware more effectively. The framework's integrated error catchall helps developers prevent website failures. Programmers can report failures even without generating additional code by using the straightforward "try/catch" function. Koa.js error handling may also be customized by developers by only changing the default configuration.

5. **It uses a context object**

Additionally, Koa.js aids developers in using Context to combine the standard response and request objects into a single object. By incorporating a variety of practical methods and assessors, the unified object facilitates the development of web apps and APIs. With several approaches for creating web apps and APIs, this context is a NodeJS object that combines the request and response into a single object.

Pros of Koa.js

1. Koa makes middleware development more enjoyable while enhancing interoperability and robustness.

2. Koa has only 550 lines of code, making it quite compact.

3. By getting rid of the confusion brought on by all those callbacks, ES6 generators will organize the code and make it easier to handle.

4. It offers an excellent user experience.

5. More comprehensible and cleaner async code.

6. No callback hell

Cons of Koa.js

1. There is a comparatively tiny open source community.

2. Incompatible with middleware in the Express style.

3. Koa uses generators that are incompatible with every other middleware for the Node.js framework.

Express.js vs Koa.js

Because both Koa.js and Express.js were created by the same team, working on them is comparable. Due to the lack of middleware and superfluous boilerplate code, Koa is a relatively light and flexible system that can be molded to meet the application requirement. The application becomes more modular when libraries are used.

Koa uses async features and promises to eliminate callback hell and streamline error handling. It exposes its own ctx.request and ctx.response objects in place of

the node's req and res objects.

While Koa lacks several framework capabilities like routing and templating, Express enhances the req and res objects in node by adding new attributes and methods.

How is Koa different from Express?

1. No callback hell
2. No boilerplate codes were used
3. Improved user experience all around
4. Using try/catch, error handling is improved
5. Koa is less dependent on middleware.
6. Koa has more modularity.
7. Routing is not offered, in contrast to Express.
8. Stream handling techniques

Creating a server using Koa.js framework

Let's get practical and learn how to build a basic server using Koa.js now that we have a better understanding of what it is and some of its capabilities.

Create a new directory for your application first, then use the terminal to browse to it and perform the following command:

> *npm init*

To create a Node.js package.

Then run to install Koa.js

> *npm i koa*

Afterward, navigate to the index file on your app's directory using your favorite code editor and write the code below to create the server.

```
const koa = require('koa')
```

Koa.js - Environment

To get started with developing using the Koa framework, you need to have Node and npm (node package manager) installed. If you dont already have these, head over to Node setup to install node on your local system. Confirm that node and npm are installed by running the following commands in your terminal.

```
$ node --version
$ npm --version
```

You should receive an output similar to –

```
v5.0.0
3.5.2
```

Please ensure your node version is above 6.5.0. Now that we have Node and npm set up, let us understand what npm is and how to use it.

Node Package Manager (npm)

npm is the package manager for node. The npm Registry is a public collection of packages of open-source code for Node.js, front-end web apps, mobile apps, robots, routers, and countless other needs of the JavaScript community. npm allows us to access all these packages and install them locally. You can browse through the list of packages available on npm at npmJS.

How to Use npm?

There are two ways to install a package using npm – globally and locally.

Globally – This method is generally used to install development tools and CLI based packages. To install a package globally, use the following command.

```
$ npm install -g <package-name>
```

Locally − This method is generally used to install frameworks and libraries. A locally installed package can be used only within the directory it is installed. To install a package locally, use the same command as above without the −**g** flag.

```
$ npm install <package-name>
```

Whenever we create a project using npm, we need to provide a package.json file, which has all the details about our project. npm makes it easy for us to set up this file. Let us set up our development project.

Step 1 − Fire up your terminal/cmd, create a new folder named hello-world and cd into it −

```
ayushgp@dell:~$ mkdir hello-world
ayushgp@dell:~$ cd hello-world/
ayushgp@dell:~/hello-world$
```

Step 2 − Now to create the package.json file using npm, use the following.

```
npm init
```

Itll ask you for the following information −

```
Press ^C at any time to quit.
name: (hello-world)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author: Ayush Gupta
license: (ISC)
About to write to /home/ayushgp/hello-world/package.json:

{
  "name": "hello-world",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Ayush Gupta",
  "license": "ISC"
}


Is this ok? (yes) yes
ayushgp@dell:~/hello-world$
```

Just keep pressing enter, and enter your name in the author name field.

Step 3 − Now we have our package.json file set up, well install Koa. To install Koa and add it in our package.json file, use the following command.

```
$ npm install --save koa
```

To confirm Koa installed correctly, run the following command.

```
$ ls node_modules #(dir node_modules for windows)
```

Tip – The **--save** flag can be replaced by **-S** flag. This flag ensures that Koa is added as a dependency to our package.json file. This has an advantage, the next time we need to install all the dependencies of our project, we just need to run the command npm install and itll find the dependencies in this file and install them for us.

This is all we need to start development using the Koa framework. To make our development process a lot easier, we will install a tool from npm, nodemon. What this tool does is, it restarts our server as soon as we make a change in any of our files, otherwise we need to restart the server manually after each file modification. To install nodemon, use the following command.

```
$ npm install -g nodemon
```

Now we are all ready to dive into Koa!

Koa.js - Hello World

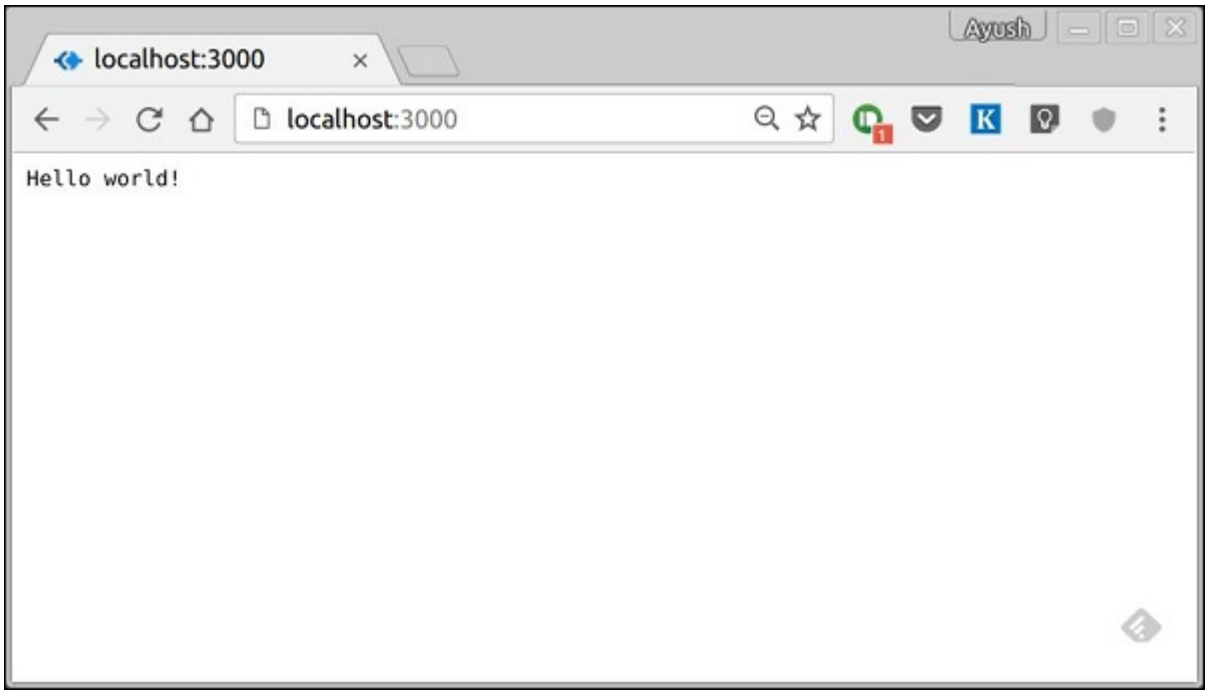
Once we have set up the development, it is time to start developing our first app using Koa. Create a new file called **app.js** and type the following in it.

```
var koa = require('koa');
var app = new koa();

app.use(function* (){
   this.body = 'Hello world!';
});

app.listen(3000, function(){
   console.log('Server running on https://localhost:3000')
});
```

Save the file, go to your terminal and type.

```
$ nodemon app.js
```

This will start the server. To test this app, open your browser and go to **https://localhost:3000** and you should receive the following message.



How This App Works?

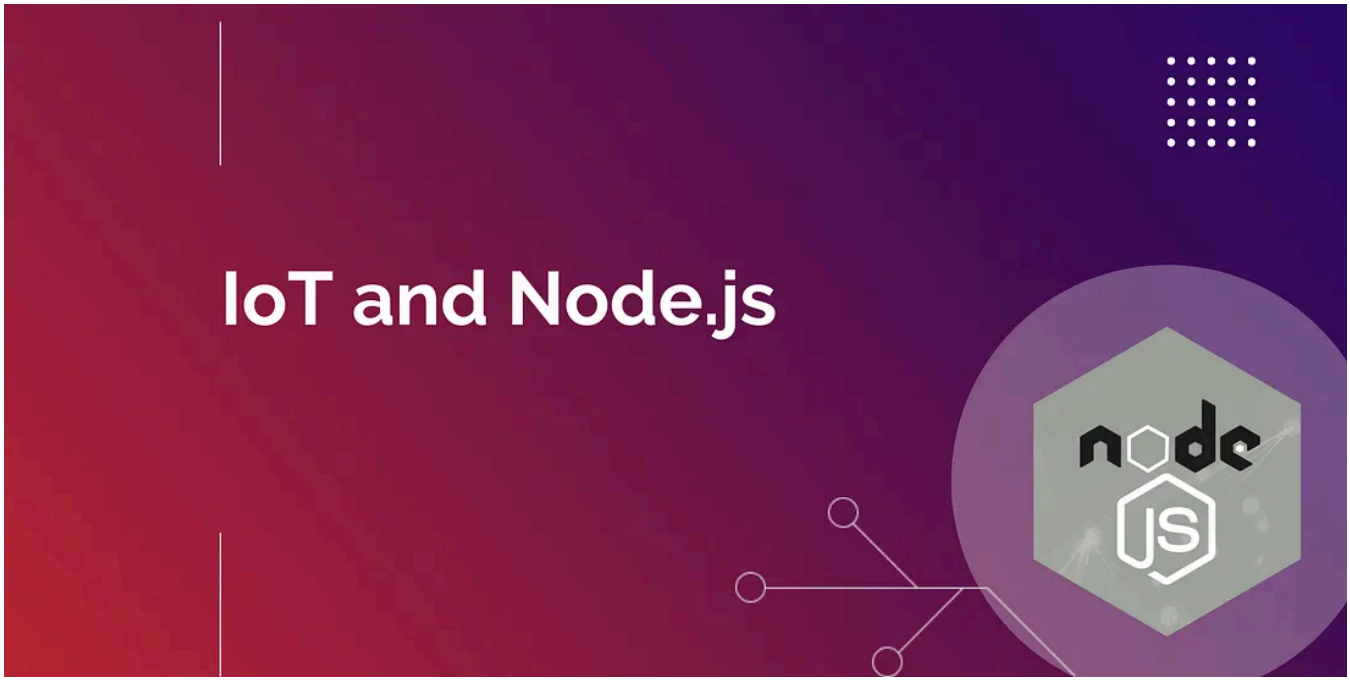
The first line imports Koa in our file. We have access to its API through the variable Koa. We use it to create an application and assign it to var app.

app.use(function) – This function is a middleware, which gets called whenever our server gets a request. We'll learn more about middleware in the subsequent chapters. The callback function is a generator, which we'll see in the next chapter. The context of this generator is called context in Koa. This context is used to access and modify the request and response objects. We are setting the body of this response to be **Hello world!**.

app.listen(port, function) – This function binds and listens for connections on the specified port. Port is the only required parameter here. The callback function is executed, if the app runs successfully.

Connecting and Controlling Devices: An In-Depth Look at Node.js in IoT

Samuelnoye · Follow
3 min read · Jan 12, 2024



The Internet of Things (IoT) has transformed the way we interact with the physical world, bringing connectivity and intelligence to everyday devices. In this article, we delve into the role of Node.js in the IoT landscape and explore a compelling case study that highlights how Node.js can be used to connect and control devices seamlessly.

Understanding the Intersection of Node.js and IoT:

Node.js, known for its lightweight and event-driven architecture, is gaining popularity in the IoT domain. Its non-blocking I/O and scalability make it an ideal

choice for handling concurrent connections and real-time communication—crucial aspects in the world of IoT.

Node.js is particularly well-suited for IoT applications due to its ability to handle asynchronous operations efficiently. It allows developers to build responsive and scalable systems, making it an excellent fit for scenarios where devices need to communicate in real time.

Case Study: Smart Home Automation with Node.js:

Let's explore a real-world case study where Node.js is employed to create a smart home automation system. Imagine a home equipped with various IoT devices, including smart lights, thermostats, and security cameras. The goal is to connect these devices, allowing users to control and monitor them remotely.

Implementation:

1. Device Communication:

Node.js acts as a central hub, facilitating communication between the IoT devices.

The MQTT (Message Queuing Telemetry Transport) protocol is used for lightweight and efficient messaging.

2. Real-time Updates:

Leveraging Node.js' event-driven nature, the system ensures real-time updates on device status changes.

WebSocket communication is employed to enable instant updates to the user interface.

3. Express for Web Server:

Node.js, with the Express.js framework, serves as the web server.

Users interact with the smart home system through a user-friendly web interface.

4. Edge Computing:

Node.js enables edge computing by running on edge devices.

This allows for quick decision-making at the device level without relying solely on cloud processing.

Benefits and Outcomes:

The integration of Node.js in this smart home automation case study yields several benefits:

1. Scalability:

Node.js handles a large number of concurrent connections efficiently, ensuring scalability as more devices are added to the system.

2. Real-time responsiveness:

Users experience real-time updates, creating a responsive and interactive smart home environment.

3. Developer Productivity:

The JavaScript ecosystem, shared between front-end and back-end development, enhances developer productivity and code maintainability.

4. Community Support:

Node.js boasts a vibrant and active community, providing extensive support, libraries, and modules for IoT development.

Conclusion:

Node.js's lightweight and event-driven architecture makes it a powerful tool for developing IoT applications. The case study presented demonstrates how Node.js can seamlessly connect and control IoT devices in a smart home setting. As the IoT landscape continues to evolve, Node.js remains a compelling choice for developers seeking efficient, scalable, and real-time solutions for their connected devices.

Nodejs JavaScript IoT Backend Development

8.4 Node and Adruino

What is Arduino with Node.js in IoT?

- Arduino is a microcontroller board that reads sensor data (like light, temperature, etc.) and controls devices (like LEDs, fans).
- Node.js is a tool running on a computer (like a laptop or Raspberry Pi) that can talk to Arduino and display or send data to the internet.
- Together, we can build cool IoT projects that connect sensors to websites or cloud.

🔧 **Example Project: Control LED from Web Browser**

✅ What it does:

Turn an LED ON/OFF from your browser using Node.js and Arduino.

🛠️ What You Need:

- Arduino Uno or Nano
- LED + Resistor (220 ohm)
- Jumper wires
- USB cable (to connect Arduino to PC)
- Node.js installed on PC
- Arduino IDE (to upload code)
- johnny-five library (for Node.js)

🔌 Circuit Diagram:

- Connect LED anode (+) to pin 13
- Connect LED cathode (-) to GND through resistor

⚙️ Step-by-Step Implementation:

1. 🧠 Upload Arduino StandardFirmata

Open Arduino IDE → Go to:

File > Examples > Firmata > StandardFirmata

- Upload this code to your Arduino board.
- This lets Node.js control the Arduino.

2. 💼 Create a Node.js Project

```
mkdir led-control
cd led-control
npm init -y
npm install johnny-five express
```

3. 🧑‍💻 Create index.js

```
const express = require('express');
const { Board, Led } = require('johnny-five');
```

```
const app = express();
const port = 3000;

const board = new Board();

board.on("ready", () => {
  const led = new Led(13); // LED connected to pin 13

  app.get('/on', (req, res) => {
    led.on();
    res.send("💡 LED is ON");
  });

  app.get('/off', (req, res) => {
    led.off();
    res.send("🔌 LED is OFF");
  });

  app.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
  });
});
```

4. ▶ Run the App

- Make sure Arduino is connected via USB.

```
node index.js
```

5. 🌐 Open in Browser

To turn LED ON: http://localhost:3000/on
To turn LED OFF: http://localhost:3000/off

8.5 Node and Raspberry Pi

Raspberry Pi with Node.js for an IoT-based application is a powerful and flexible approach. Raspberry Pi provides the hardware interface and control, while Node.js enables asynchronous and event-driven server-side logic — ideal for handling IoT tasks like sensor data processing, real-time communication, and cloud integration.

Why Node.js on Raspberry Pi for IoT?

- Lightweight and Fast: Node.js uses the V8 engine for fast execution.
- Non-blocking I/O: Perfect for handling multiple device inputs/outputs.
- Large Package Ecosystem: NPM offers thousands of packages for sensors, networking, etc.
- Web Integration: Easily create web servers or REST APIs for your IoT data.

Example: Temperature Monitoring System using DHT11 Sensor

📌 Goal : Build a system that reads temperature and humidity data from a DHT11 sensor using Raspberry Pi and displays it on a web dashboard using Node.js.

🧰 Hardware Required :

- Raspberry Pi (any model with GPIO support)
- DHT11 sensor
- Jumper wires
- Breadboard

🧑‍💻 Step-by-Step Implementation:

1. Connect the DHT11 Sensor to Raspberry Pi

DHT11 Pin	Connect To Raspberry Pi
VCC	5V
GND	GND
Data	GPIO4 (Pin 7)

2. Setup Raspberry Pi

- Install Node.js:

```
sudo apt update
sudo apt install nodejs npm
```

- Check versions:

```
node -v
npm -v
```

3. Install Required Node Packages

```
npm init -y
npm install express onoff node-dht-sensor
```

4. Node.js Code (index.js)

```
const express = require('express');
const sensor = require('node-dht-sensor');

const app = express();
const port = 3000;

sensor.initialize(11, 4); // DHT11, GPIO4
```

```
app.get('/data', (req, res) => {
  const readout = sensor.read();
  res.json({
    temperature: readout.temperature.toFixed(1),
    humidity: readout.humidity.toFixed(1)
  });
});

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

5. Run the Application

```
sudo node index.js
```

Open your browser and visit: **Fehler! Linkreferenz ungültig.**
You'll see live temperature and humidity data in JSON format.

📝 Arduino – Detailed Note (For 7 Marks)

🔹 Introduction:
Arduino is an open-source electronics platform based on easy-to-use hardware and software. It is mainly used to build electronic projects, especially in IoT, robotics, and automation.

🔹 What is Arduino?

- Arduino is a microcontroller board, not a full computer like Raspberry Pi.
- It can read inputs (like light, temperature, motion) and control outputs (like LEDs, motors, buzzers).
- It is programmed using the Arduino IDE with simple C/C++ language.

🔹 Popular Arduino Boards:

Board Name	Description
Arduino Uno	Most popular and beginner-friendly
Arduino Nano	Small size, same power as Uno
Arduino Mega	More memory and input/output pins
Arduino Leonardo	Can act like a keyboard/mouse

🔹 Main Components of Arduino Board:

1. Microcontroller (e.g., ATmega328P) – The brain of Arduino.

2. Digital I/O Pins – To connect LEDs, switches, etc.
3. Analog Input Pins – To read sensor values.
4. Power Port (USB) – To power and upload code.
5. Reset Button – To restart the program.
6. Voltage Regulator – Maintains stable voltage.

🔹 Programming Arduino:
- Use Arduino IDE on a computer.
- Write code in C/C++ language.
- Upload the code to the board via USB cable.
- Once uploaded, the board runs the code without needing a computer.

🔹 Applications of Arduino:
1. IoT Projects – Smart home, weather monitoring.
2. Robotics – Line follower robot, obstacle avoider.
3. Automation – Auto lights, temperature control.
4. Sensors & Actuators – Read and control real-world data.
5. Educational Projects – For students and hobbyists.

🔹 Advantages of Arduino:
- Easy to use and beginner-friendly
- Open-source and low cost
- Large online community and tutorials
- Compatible with many sensors and modules

🔹 Limitations of Arduino:
- Not suitable for heavy computing tasks
- No built-in Wi-Fi (unless using modules)
- Limited memory compared to full computers

🔚 Conclusion:
Arduino is a powerful, flexible, and affordable platform for learning electronics, building smart devices, and exploring IoT applications. It is widely used by students, engineers, and hobbyists around the world.

📝 Raspberry Pi – Detailed Note (For 6 Marks)

🔹 Introduction:
Raspberry Pi is a small, low-cost, single-board computer developed by the Raspberry Pi

Foundation in the UK. It is used to learn programming, electronics, and build IoT-based smart projects.

🔹 Features of Raspberry Pi:

1. Compact Size: It is as small as a credit card.
2. Low Power Consumption: Works on a 5V power supply (like a phone charger).
3. USB Ports: Connects to keyboard, mouse, pen drives, etc.
4. HDMI Port: Connects to monitor or TV.
5. SD Card Slot: Used for installing the operating system and storing files.
6. GPIO Pins: Used to connect sensors, LEDs, motors, and other electronic components.
7. Internet Support: Has Wi-Fi and Ethernet port for internet connectivity.
8. Audio and Camera Support: Has a 3.5mm audio jack and camera interface.

🔹 Operating System:

- Raspberry Pi mainly uses Raspberry Pi OS (earlier called Raspbian), which is a version of Linux.
- Other OS options: Ubuntu, Windows IoT Core, and more.
- The OS is stored and booted from a microSD card.

🔹 Common Uses of Raspberry Pi:

1. Learning Programming (Python, Java, C++, etc.)
2. IoT Projects (Home automation, weather stations, etc.)
3. Robotics
4. Media Center (using software like Kodi)
5. Web Server Hosting
6. Security Systems (CCTV camera setups)
7. Game Emulation Console

🔹 Advantages:

- Affordable and easy to use
- Great for beginners and students
- Huge online community and support
- Works with many sensors and devices
- Portable and energy efficient

🔹 Limitations:

- Not as powerful as full desktop computers
- May heat up if used for heavy tasks
- Needs technical knowledge for complex projects

⇇ Conclusion:
Raspberry Pi is a powerful mini computer used widely in education, DIY projects, IoT, and automation. Its low cost and flexibility make it perfect for learning and building innovative technology-based systems.